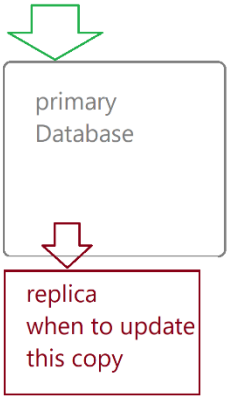
AZ-204 Class Notes 202X

Contents

Links for AZ-204 – 05/06/2024 – 12/03/2025

1. Course AZ-204T00-A: Developing Solutions for Microsoft Azure - Training | Microsoft Learn
   a. Official Book
2. GitHub - MicrosoftLearning/AZ-204-DevelopingSolutionsforMicrosoftAzure: AZ-204: Developing solutions for Microsoft Azure
   a. Lab files & Instructions
3. Microsoft Certified: Azure Developer Associate - Certifications | Microsoft Learn
   a. AZ-204 Exam Link
4. Study guide for Exam AZ-204: Developing Solutions for Microsoft Azure | Microsoft Learn

Consistency Level	Description
Strong saved	When writes are performed on your primary database, the operation is replicated to the replica instances. The write operation is committed (and visible) on the primary only after it has been committed and confirmed by all replicas. record or row
Bounded Staleness	Like the Strong level with the difference that you can configure how stale documents can be within replicas. Staleness is the quantity of time (or the version count) a replica document can be behind the primary document.
Session	This level guarantees that all read and write operations are consistent within a user session. Within the user session, all reads and writes are monotonic and guaranteed to be consistent across primary and replica instances.
Consistent Prefix	This level has loose consistency but guarantees that when updates show up in replicas they will show up in the correct order (that is, as prefixes of other updates) without any gaps.
Eventual	This level has the loosest consistency and essentially commits any write operation against the primary immediately. Replica transactions are asynchronously handled and will eventually (over time) be consistent with the primary. This tier has the best performance, because the primary database does not need to wait for replicas to commit to finalize its transactions.



Code – Dotnet – Az CLI – PowerShell – C# - Syntax – Script - Snippets

5. dotnet command - .NET CLI | Microsoft Learn
6. Azure Command-Line Interface (CLI) documentation | Microsoft Learn
7. PowerShell Documentation - PowerShell | Microsoft Learn
8. Login dup
9. 

10. endList

Syntax: What Gets Typed & Special Characters

In C#, the **<BlobContainerClient>** part in private async Task<BlobContainerClient> is a **generic type parameter**.

Breaking it down:

- Task<T> is a **generic class** in the .NET System.Threading.Tasks namespace that represents an **asynchronous operation** that returns a result of type T.
- In your example:
- private async Task<BlobContainerClient> SomeMethodName()

This means:

- The method is private (accessible only in this class).
- It's async (can use await and returns asynchronously).
- It returns a Task<BlobContainerClient>, meaning:
  - The method runs asynchronously.
  - When awaited, it will eventually return a BlobContainerClient object.

So what are < > in this context?

They're **type parameters** used in **generics**, telling the compiler what type to substitute for T.

A few more examples for clarity:

Syntax	Meaning
List<string>	A list of strings
Dictionary<int, string>	A dictionary with int keys and string values
Task<int>	An async method returning an int
IEnumerable<MyClass>	An enumerable collection of MyClass instances

Let me know if you want a visual or deeper dive into generics!

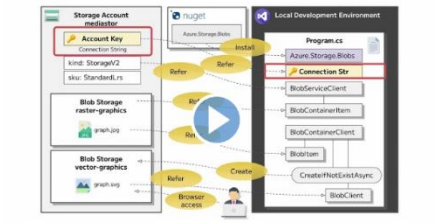
```
private async Task<BlobContainerClient> GetCloudBlobContainer(string containerName)
```

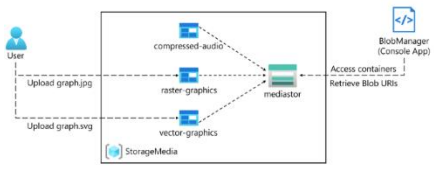
private async Task<BlobContainerClient>
- what are the greater than & less than symbols in c# syntax

The ellipsis symbol (...) is **a punctuation mark consisting of three periods (or dots) used to indicate an omission of words, a pause, or a trailing off of thought**. It can also indicate an unfinished sentence or a trailing off into silence, according to Wikipedia. In some contexts, particularly in quoted material, it signifies that some text has been omitted.

AZ-204 Labs Table of Contents & Tim's Notes

1. Module 01: Creating Azure App Service Web Apps
    a. Create a web application on Azure by using the PaaS model. After the web application is created, you will learn how to upload existing web application files by using the Apache Kudu zip deployment option.
    b. Imgapi
    c. imgweb
2. Module 02: Implement Azure Functions vs. Durable w/Orchestration
    a. Create a simple Azure function that echoes text that is entered and sent to the function by using HTTP POST commands.
    b. Connect to a storage account that you create and return the contents of a file that is stored in the Azure storage account
    c. Execute on a fixed schedule. – CRON job is a timer or an Agent in MS Windows
    d. The function will write a message to a log each time the schedule is triggered.
    e. Create a new local Azure Functions project in the current directory using the **dotnet-isolated** runtime:
    f. Develop Azure Functions locally using Core Tools | Microsoft Learn
    g. Create your first function in the Azure portal | Microsoft Learn
    h. Create a C# function using Visual Studio Code - Azure Functions | Microsoft Learn
    i. Azure Functions Core Tools reference | Microsoft Learn
    j. Create a function that integrates with Azure Logic Apps | Microsoft Learn
    k. Test web APIs with the HttpRepl | Microsoft Learn
    l. funclogic – Function App
3. Module 03: Develop solutions that use blob storage
    a. Use the Azure Storage SDK to access Azure Storage containers within a C# application – C.R.U.D. Ops.
    b. Use the Azure Storage SDK to access Azure Storage containers within a C# application.
    c. Access metadata and expose URI information to gain access to the contents of the containers in the storage account

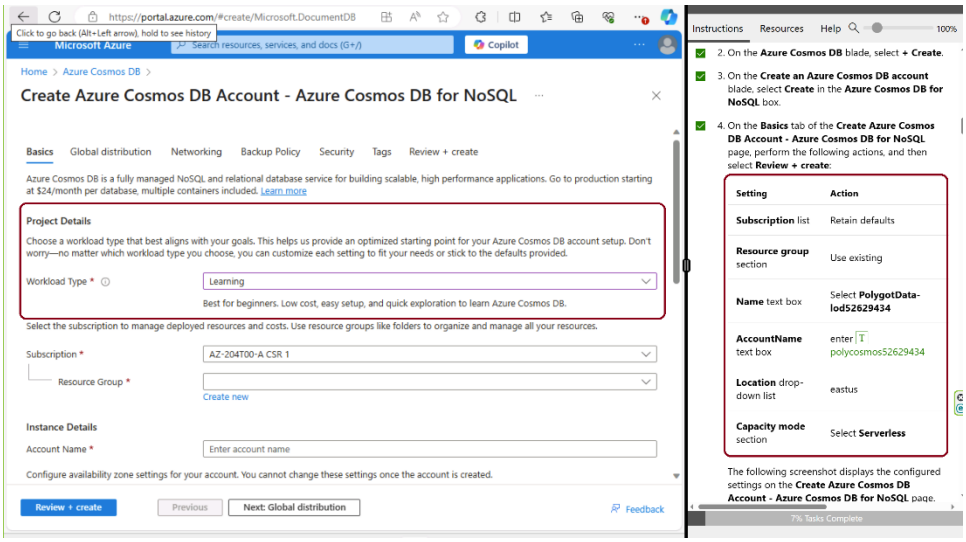




    d.
    e. BlobManager - console
    f. Explore Azure Storage security features - Training | Microsoft Learn
    g. Manage the Azure Blob storage lifecycle - Training | Microsoft Learn – access tiers

h. Work with Azure Blob storage - Training | Microsoft Learn – methods

Cosmos DB



4. Module 04: Develop solutions that use Cosmos DB storage (aka Polyglot)
    a. Access the data in Cosmos DB through a user-friendly interface, you will implement a .NET solution that accesses and displays the data from Cosmos DB in a web browser.
        i. AdventureWorks.Upload
        ii. AdventureWorks.Web
        iii. AdventureWorks.Context
    b. Exercise: Create Azure Cosmos DB resources by using the Azure portal - Training | Microsoft Learn
    c. Work with Azure Cosmos DB - Training | Microsoft Learn

Change Feed

Is Azure Cosmos DB Change Feed a traffic management tool?

Not really. **Change Feed** isn't about routing or balancing traffic like a load balancer or API gateway. Instead, it's a **data event stream** feature. It continuously captures and exposes changes (inserts and updates) in your Cosmos DB container in the order they occur. Think of it as a **real-time log of what changed in your database**, which you can consume to trigger downstream actions.

Why is it useful in the IoT world?

Here's the simple reason:

IoT devices generate **tons of telemetry data**—temperature readings, sensor states, GPS positions, etc. You often need to **react to changes immediately** (e.g., alert when a sensor crosses a threshold, update dashboards, or feed analytics pipelines). Instead of polling the database constantly (which is inefficient and costly), **Change Feed gives you a push-like mechanism**:

- **Real-time processing**: As soon as new data arrives, you can process it.

- **Scalable event-driven architecture**: Hook it up to Azure Functions, Event Hubs, or Stream Analytics for automation.
- **Reliable ordering**: Events are delivered in the order they were written, which is critical for time-series IoT data.

✅ **Example IoT use case**:
Imagine thousands of smart meters sending energy usage data to Cosmos DB. With Change Feed:

- You can trigger an Azure Function whenever usage spikes.
- Push aggregated data to a dashboard in near real-time.
- Feed machine learning models for predictive maintenance.

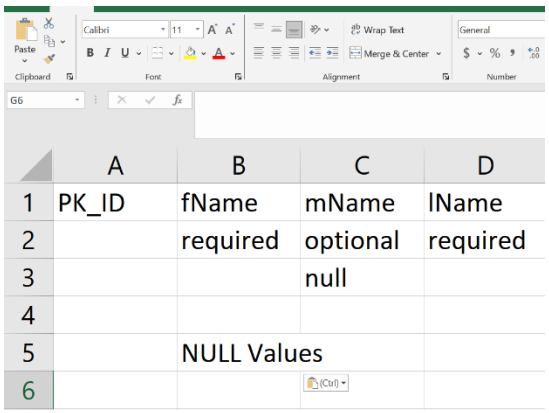
---

d. Working with the change feed - Azure Cosmos DB | Microsoft Learn
    i. Change feed in Azure Cosmos DB is a persistent record of changes to a container in the order they occur.

How do relationships get created?

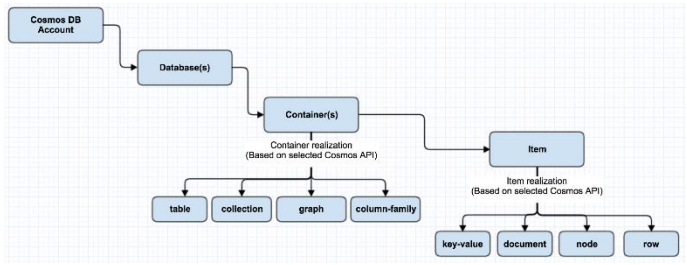
e. When a container is created, you need to supply a partition key.
    i. The value of this property is then used to route data to the appropriate partition to be written, updated, or deleted.
    ii. Items in a container are divided into distinct subsets called logical partitions.
f. Partitions
    i. Scale individual containers in a database.
g. Logical partitions are formed based on the value of a partition key
    i. All items are grouped by the partition key value in each item in a container.
    ii. Each item in a container has an item ID (unique within a logical partition).
    iii. Combining the partition key and the item ID creates the item's index and uniquely identifies the item.
    iv. The partition key can also be used in the WHERE clause in queries for efficient data retrieval.
    v. Partitioning and horizontal scaling - Azure Cosmos DB | Microsoft Learn
h. Databases, containers, and items - Azure Cosmos DB | Microsoft Learn

i.

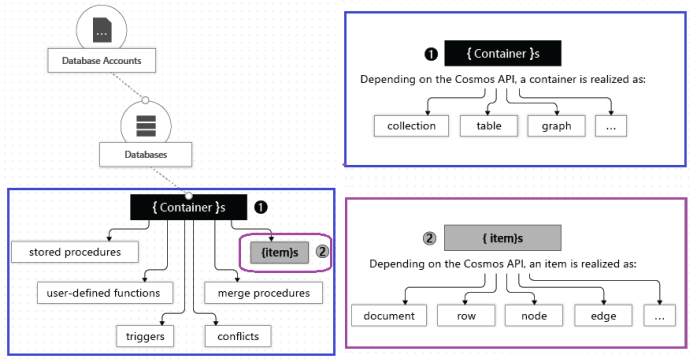


	A	B	C	D
1	PK_ID	fName	mName	lName
2		required	optional	required
3			null	
4				
5		NULL Values		
6				

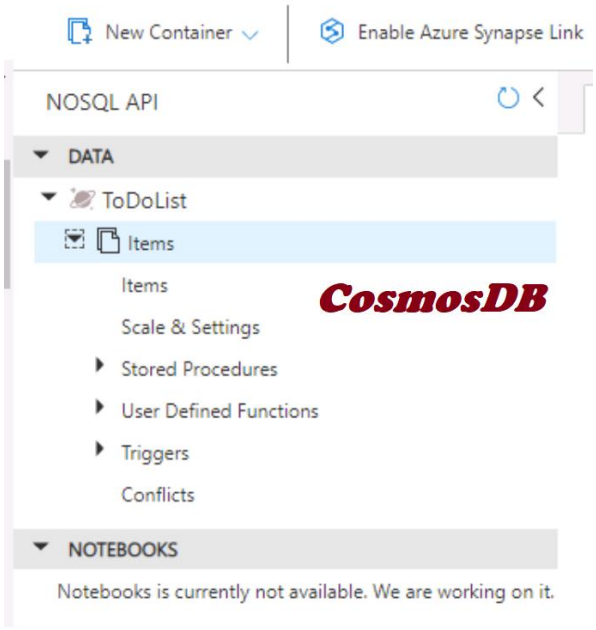
j.

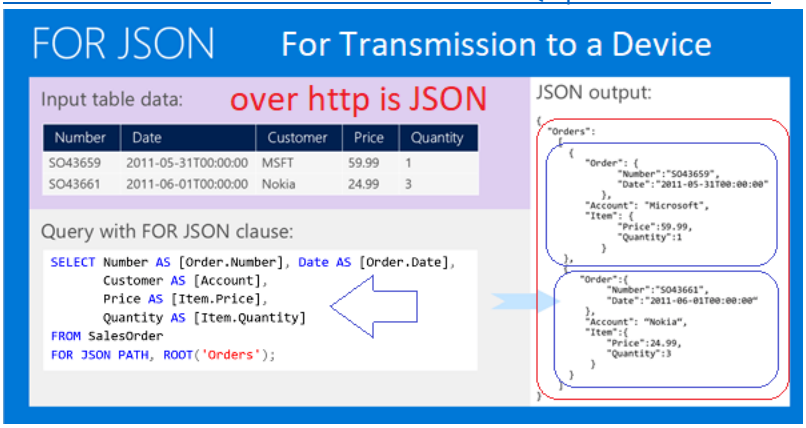


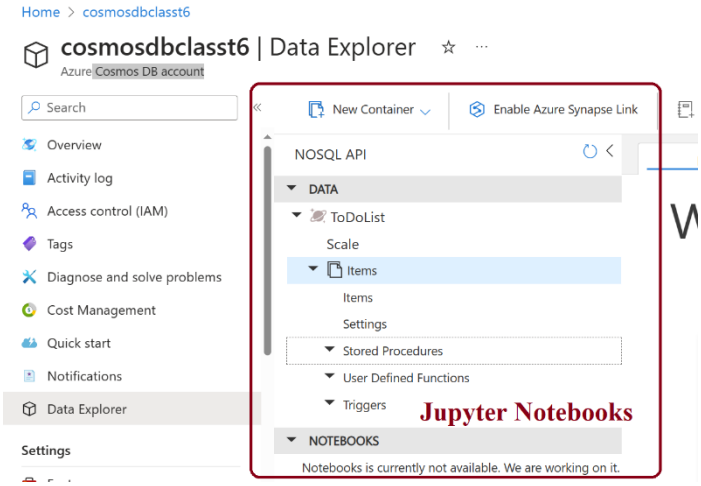
k.



l.

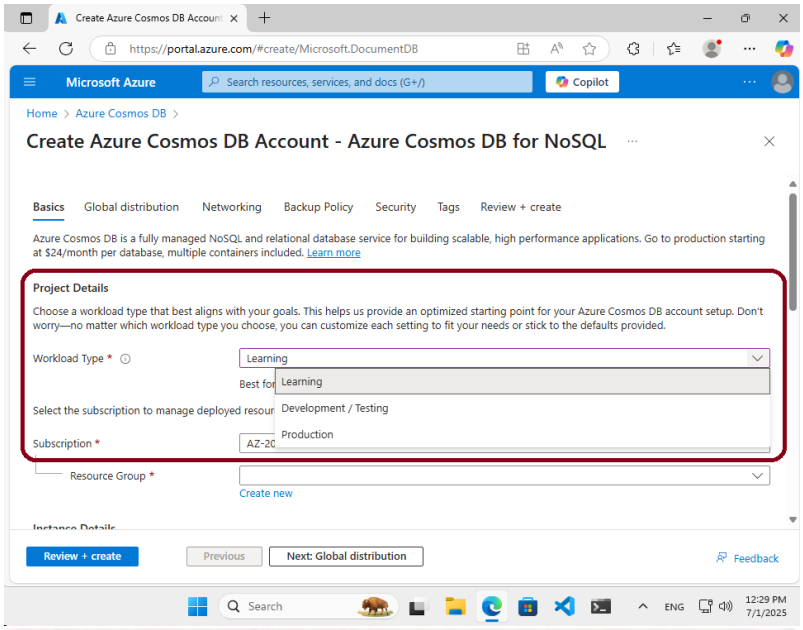


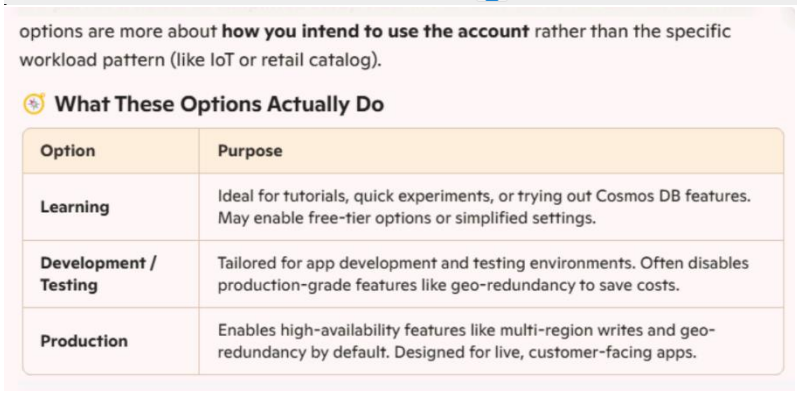
m. *Quickstart - Create Azure Cosmos DB resources from the Azure portal | Microsoft Learn - simple
n. Tutorial: Query data - Azure Cosmos DB for NoSQL | Microsoft Learn
    i. Uses document and cross product
o. NoSQL queries - Azure Cosmos DB for NoSQL | Microsoft Learn
p. Work with JSON - Azure Cosmos DB for NoSQL | Microsoft Learn
q. 

r. Write stored procedures, triggers, and UDFs in Azure Cosmos DB | Microsoft Learn
    i. Stored procedures are written using JavaScript, and they can create, update, read, query, and delete items inside an Azure Cosmos DB container.
    ii. DP-420: Designing and Implementing Cloud-Native Applications Using Microsoft Azure Cosmos DB (microsoftlearning.github.io)
s. Tutorial: Create a Jupyter Notebook in Azure Cosmos DB for NoSQL to analyze and visualize data (preview) | Microsoft Learn
t. 

u. Azure Cosmos DB, RU/s stands for Request Units per second. It represents the throughput capacity of a database or container and is a measure of the computational resources (CPU, IO, memory) required to perform database operations. When you provision throughput, you're essentially reserving a specific number of RUs per second for your workload.

Cosmos Workload Type

v. 

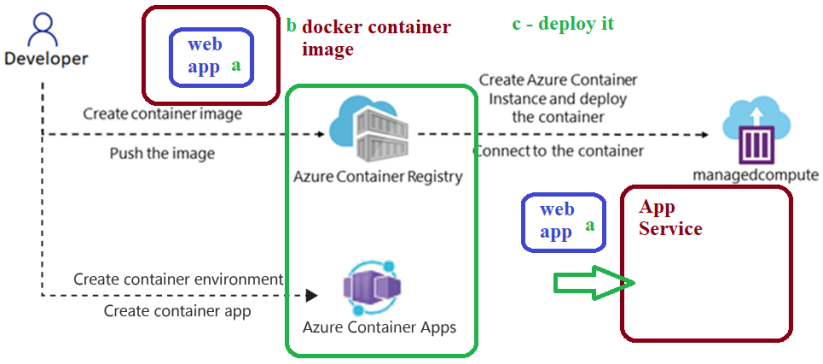
w. 

options are more about **how you intend to use the account** rather than the specific workload pattern (like IoT or retail catalog).

What These Options Actually Do

Option	Purpose
Learning	Ideal for tutorials, quick experiments, or trying out Cosmos DB features. May enable free-tier options or simplified settings.
Development / Testing	Tailored for app development and testing environments. Often disables production-grade features like geo-redundancy to save costs.
Production	Enables high-availability features like multi-region writes and geo-redundancy by default. Designed for live, customer-facing apps.

5. Module 05: Implement IaaS solutions
    a. Console application to display a machine's current IP address.
        i. Ipcheck
        ii. containerapp
    b. Add **Dockerfile** file to the application so that it could be converted into a Docker container image.
    c. Deployed the container image to Container Registry.
    d. [AZ-204: Developing solutions for Microsoft Azure (microsoftlearning.github.io)]
    e. [Build a containerized web application with Docker - Training | Microsoft Learn]
    f. [ACR Tasks overview - Azure Container Registry | Microsoft Learn]



g.

instead of App Service - Container ☐

Docker - software that packages an app for deployment

Azure Container Registery
Azure Container Manageme

Continuous Integration/Con
Deployment (CI/CD)

chat app

deploy from code on a schedule w/ DevOps

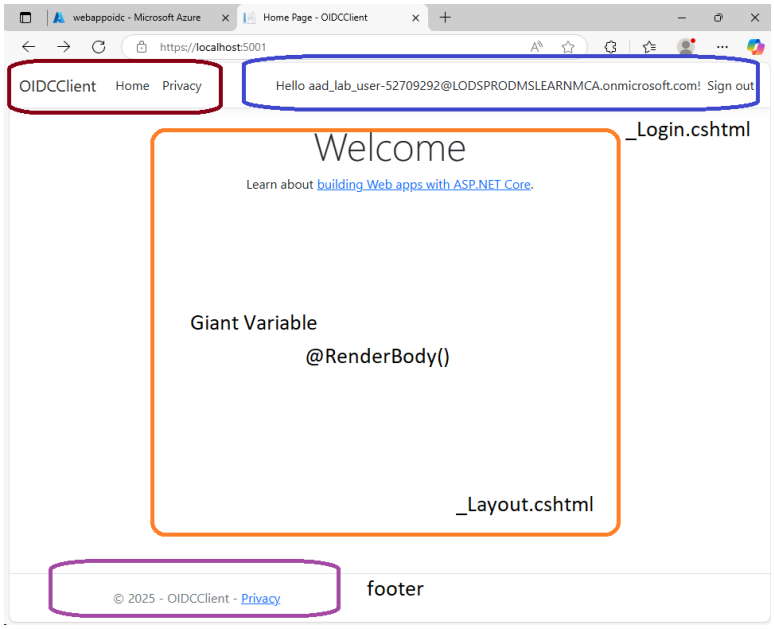
h.

6. Module 06: Implement user authentication and authorization
    a. Register an application in Microsoft **Entra ID**, add a user, and then test the user's access to the application to validate that Entra ID can secure it.
        i. OIDCClient MVC app
    b. You will also use the **Graph Explorer** tool to build and test requests against the Graph API for an Entra ID user account.
    c. [AZ-204: Developing solutions for Microsoft Azure (microsoftlearning.github.io)]
    d. [Explore the Microsoft identity platform - Training | Microsoft Learn]

Parameter	Description
/f	Forces deletion of read-only files.
/s	Deletes specified files from the current directory and all subdirectories. Displays the names of the files as they are being deleted.
/q	Specifies quiet mode. You are not prompted for delete confirmation.

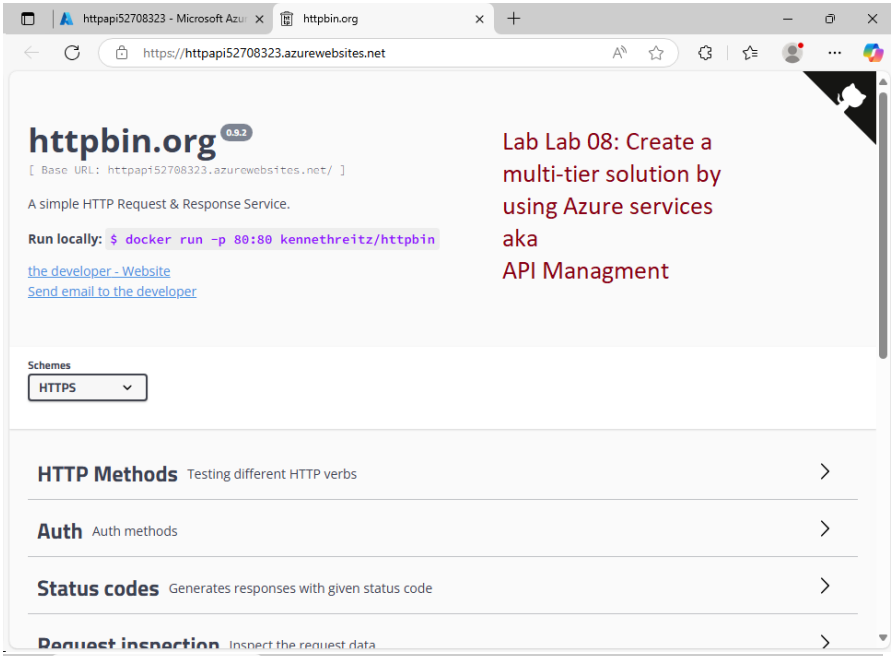
    e.
    f. [Discover Microsoft Graph - Training | Microsoft Learn]
7. Module 07: Implement secure cloud solutions
    a. Create a storage account and an Azure Function app that will access the storage account.
        i. func init
    b. Demonstrate the secure storage of connection string information.
    c. Provision a Key Vault resource and manage the appropriate secrets to store the connection string information.

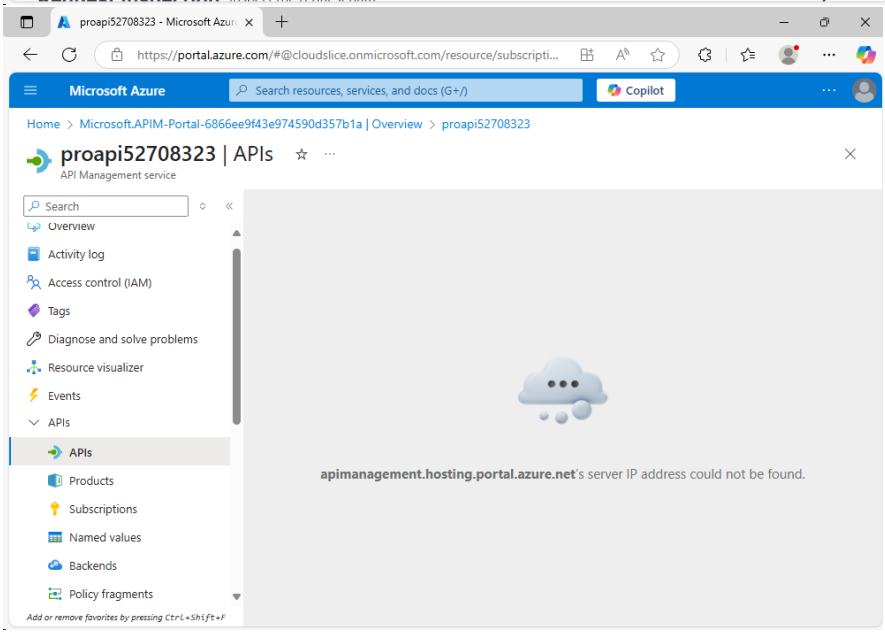
d. [AZ-204: Developing solutions for Microsoft Azure (microsoftlearning.github.io)](#)

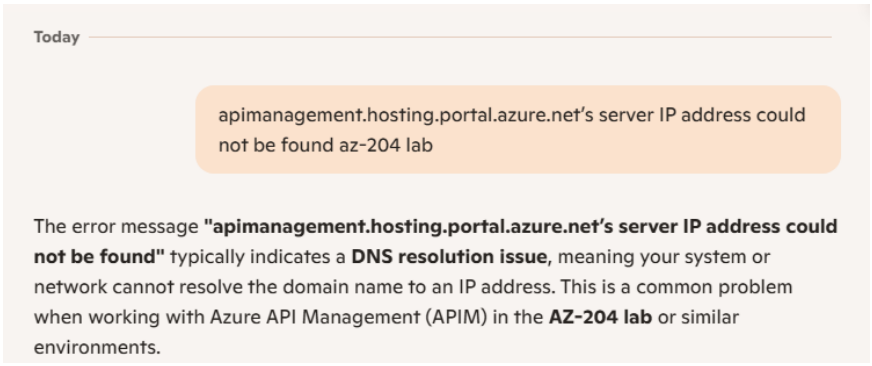


e.

8. Module 08: Implement API Management
    a. Define a new API
    b. Create a containerized (Docker) application to host a web app on Azure, as the source of information for the API.
        i. kennethreitz/httpbin:latest
        ii. [kennethreitz/httpbin - Docker Image | Docker Hub](#)
        iii. Httpbin.org
        iv. [Releases · Kenneth Reitz / httpbin · GitLab](#)
    c. You will then build an API proxy using the Azure API Management capabilities to expose and test your APIs.
    d.

e. 

f. 

g. 

APPLIES TO: All API Management tiers

h. In Azure API Management, a product contains one or more APIs, a usage quota, and the terms of use. After a product is published, developers can subscribe to the product and begin to use the product's APIs.

i. https://learn.microsoft.com/en-us/azure/api-management/api-management-howto-add-products?WT.mc_id=Portal-fx&tabs=azure-portal&pivots=interactive

j.

🔹 What the Forward Slash `/` Denotes in a URL

Position in URL	Meaning
After the domain (e.g., `https://example.com/`)	Denotes the **root** of the website or web application
Between path segments (e.g., `/products/shoes`)	Separates **hierarchical levels** in the path
At the end of a path (e.g., `/about/`)	Often denotes a **directory** rather than a file (though this depends on server configuration)
Double slashes after protocol (e.g., `https://`)	Separates the **protocol** from the rest of the URL
In query strings or fragments	Rarely used here; more common delimiters are `?`, `&`, and `#`

k.

🧭 Examples

- `https://example.com/` → The root of the site.
- `https://example.com/blog/` → The `/blog/` directory or route.
- `https://example.com/blog/post-1` → A specific resource under `/blog/`.

🧩 Special Notes

- A single `/` by itself in a GET request (like `GET / HTTP/1.1`) refers to the **root path** of the server.
- In RESTful APIs, slashes define **resource hierarchies**:
  - `GET /users/123/orders` → Get orders for user with ID 123.

l.

💉 Frontend vs Backend Testing: What They Show

Aspect	Frontend Testing	Backend Testing
What it tests	User interface (UI), layout, buttons, forms, responsiveness	Server logic, APIs, databases, authentication, business rules
Who sees it	End users (what they interact with)	Developers and systems (behind the scenes)
Tools used	Selenium, Cypress, Playwright	Postman, JMeter, SQL tools
Focus	Usability, visual correctness, cross-browser/device compatibility	Data integrity, performance, security, logic correctness
Example	Does the "Submit" button work and look right?	Does the form actually save data to the database correctly?

m.

Stage	Direction	Purpose	Example Use Case
Inbound	Request	Preprocess client request	Add auth headers, validate tokens
Backend	Request	Configure backend call	Change backend URL, set timeout
Outbound	Response	Postprocess backend response	Format response, redact data

n.

the terms **inbound**, **backend**, and **outbound** refer to different stages of the **API request/response pipeline**. Understanding these stages helps you apply the right logic or transformation at the right time.

The API Request Lifecycle

When a client sends a request to an API through APIM, it flows through these stages:

```
Client → Inbound → Backend → Outbound → Client
```

o.

Inbound

- **What it is**: The stage where the request **enters** the API gateway from the client.
- **Purpose**: Modify or inspect the request **before** it reaches the backend.
- **Common uses**:
  - Authentication and authorization
  - Rate limiting
  - Header or query parameter transformation
  - Logging or telemetry

Example:

```xml
<inbound>
  <set-header name="x-api-key" exists-action="override">
    <value>my-secret-key</value>
  </set-header>
</inbound>
```

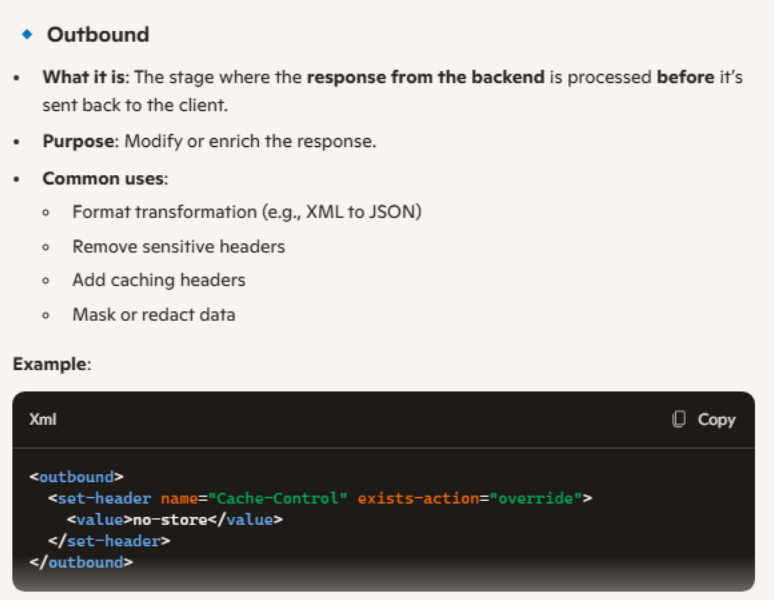
p.

Backend

- **What it is**: The stage where the request is **prepared to be sent** to the actual backend service.
- **Purpose**: Configure how the gateway communicates with the backend.
- **Common uses**:
  - Set timeout or retry policies
  - Change the backend URL dynamically
  - Add backend-specific headers

Example:

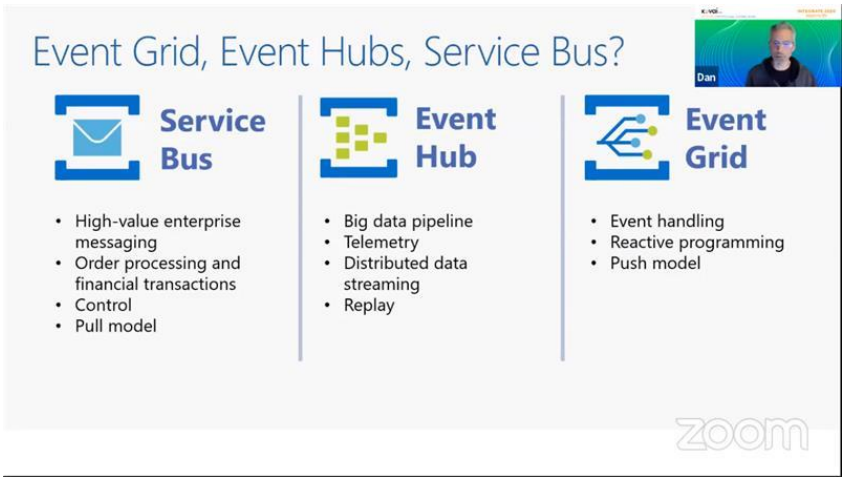
```xml
<backend>
  <set-backend-service base-url="https://my-backend.com/api" />
</backend>
```



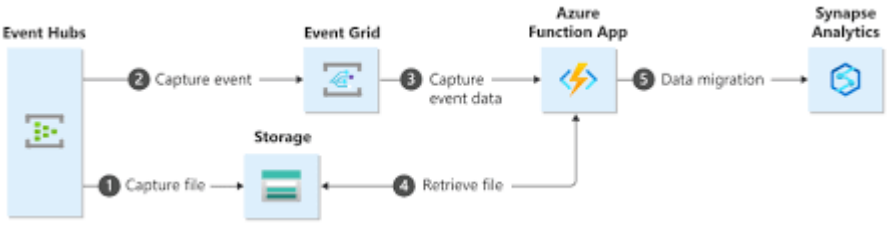
q.

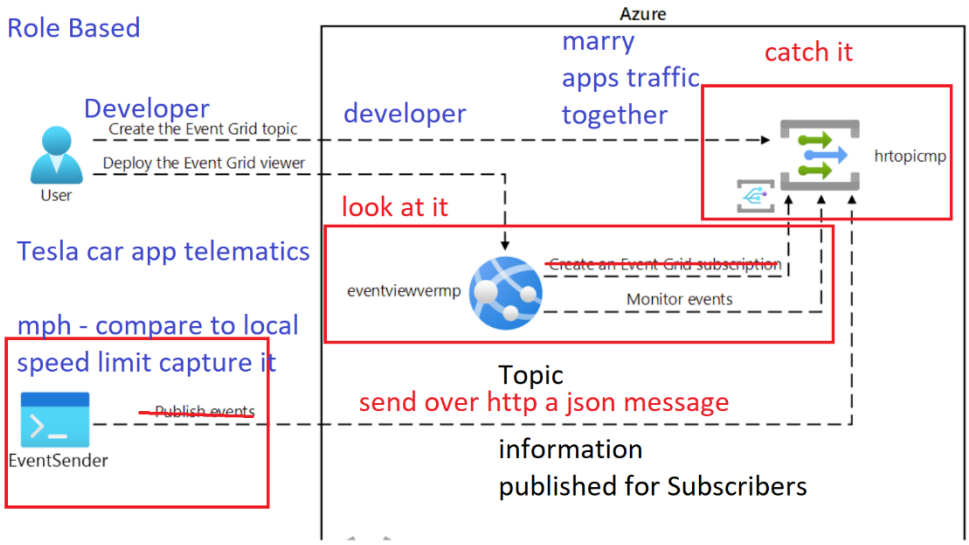
r. [AZ-204: Developing solutions for Microsoft Azure (microsoftlearning.github.io)]

9. Module 09: Develop event-based solutions
    a. [Azure messaging services documentation | Microsoft Learn]
        i. Event Grid pushes the ingested data to the subscribers whereas Event Hubs makes the data available in a pull model
    b. [Overview - Azure Event Grid | Microsoft Learn]
    c. Start with a proof-of-concept web app, hosted in a container, that will be used to subscribe to your **Event Grid**.
        i. microsoftlearning/azure-event-grid-viewer:latest
        ii. EventPublisher console
    d. This app will allow you to submit events and receive confirmation messages that the events were successful.



    e.
    f. [Tutorial: Send Event Hubs data to data warehouse - Event Grid - Azure Event Grid | Azure Docs]

g. 

h. 

i. Use cases for using Azure Event Grid - Azure Event Grid | Microsoft Learn
j. AZ-204: Developing solutions for Microsoft Azure (microsoftlearning.github.io)
k. Explore Azure Event Grid - Training | Microsoft Learn
l. Azure Event Hubs – data streaming platform with Kafka support - Azure Event Hubs | Microsoft Learn
m. Exercise: Send and receive message from a Service Bus queue by using .NET. - Training | Microsoft Learn
n. Quickstart: Send custom events to Azure Function - Event Grid - Azure Event Grid | Microsoft Learn
o. Azure Event Hubs – data streaming platform with Kafka support - Azure Event Hubs | Microsoft Learn
    i. Azure Event Hubs is a cloud native data streaming service that can stream millions of events per second
    ii. A key difference between Event Grid and Event Hubs is in the way event data is made available to the subscribers.
    iii. Event Grid pushes the ingested data to the subscribers
    iv. Event Hubs makes the data available in a pull model. As events are received, Event Hubs appends them to the stream.
    v. Asynchronous messaging options - Azure Architecture Center | Microsoft Learn
p. Event Grid Topics
    i. Send events to subscribers (push delivery) or subscribers can connect to Event Grid to read events (pull delivery).

ii. MQTT, which stands for Message Queuing Telemetry Transport, is a lightweight, open, and simple messaging protocol designed for constrained devices and low-bandwidth, high-latency, or unreliable networks.
iii. It operates on a publish-subscribe model, where devices (publishers) send messages to a central broker, and other devices (subscribers) receive those messages based on their subscriptions.
iv. MQTT is widely used in the Internet of Things (IoT) for connecting devices and enabling communication between them.

MQTT vs HTTP: Protocol Comparison

Feature	MQTT	HTTP
Type	Message-based publish/subscribe	Request/response
Transport	TCP (often on port 1883), or WebSockets	TCP (usually over port 80/443)
Communication Model	**Asynchronous**, many-to-many (pub/sub)	**Synchronous**, one-to-one (client-server)
Overhead	Very low (ideal for constrained devices)	Higher (verbose headers, full HTTP stack)
Text/Binary	Binary protocol	Text-based (headers, body, etc.)
Data Format	Arbitrary (e.g., JSON, binary blobs)	Typically HTML/JSON/XML
Statefulness	Long-lived connections (persistent sessions)	Stateless (each request is new)
Use Case Fit	IoT, telemetry, sensors, embedded devices	Web browsing, APIs, standard clients

v.
vi. Azure-Samples/MqttApplicationSamples: Samples implementing common PubSub patterns for Edge and Cloud Brokers
vii. Create, view, and manage Azure Event Grid namespaces - Azure Event Grid | Microsoft Learn
viii. A namespace in Azure Event Grid is a logical container for one or more topics, clients, client groups, topic spaces, and permission bindings.
ix. A Topic is a logical channel or name used to group events.
x. Think of it as a "category" or "stream" where events are published, and subscribers can listen.
xi. Acts as a routing address for published events..
xii. Allows event publishers to send messages to a known location
xiii. The event subscription is what connects a handler (like a function or webhook) to a specific topic.
xiv. Subscribers then attach to that topic to receive only the events published to it.
xv. Schema Format

xvi.

First: Yes, **events are always JSON-formatted**

But... **the *structure* of the JSON** changes depending on the **schema** you choose.

The 3 Event Schemas in Azure Event Grid

Schema	Structure	Who defines it?	Best used when...
Event Grid Schema	Native to Azure	Microsoft	You're working within Azure's ecosystem
CloudEvents v1.0	CNCF standard	CloudEvents community	You want portability across clouds/systems
Custom Schema	You define it	You	You have legacy or domain-specific formats

xvii.

1. Event Grid Schema (Default)

Azure-native format. Fields include:

```json
{
  "id": "123",
  "eventType": "Contoso.Items.ItemReceived",
  "subject": "/items/blue-widget",
  "eventTime": "2025-07-07T18:30:52.123Z",
  "data": {
    "itemSku": "SKU12345"
  },
  "dataVersion": "1.0",
  "metadataVersion": "1"
}
```

2. CloudEvents v1.0

Industry standard schema — useful for cross-cloud or Kubernetes environments:

```json
{
  "specversion": "1.0",
  "type": "com.contoso.items.received",
  "source": "/items",
  "id": "123",
  "time": "2025-07-07T18:30:52.123Z",
  "datacontenttype": "application/json",
  "data": {
    "itemSku": "SKU12345"
  }
}
```

q. Event Grid Viewer

i.

The **Azure Event Grid Viewer** is a **sample web app** (typically hosted on Azure App Service) that:

- Displays incoming **event messages** from an Event Grid topic or domain
- Is often used to **visualize or debug** the event flow
- Is **read-only** — it does not send or generate events, just listens

ii.

How it works

1. You deploy the Event Grid Viewer (a basic web app).
2. You create an **event subscription** with the **viewer URL** as the endpoint.
3. It listens and renders incoming event data (as JSON in the UI).

10. Module 10: Develop message-based solutions
    a. Create a proof of concept for this scenario by employing an **Azure Service Bus Queue**.
    b. Create a .NET Core project that will publish messages to the system,
    c. A second .NET Core application that will read messages from the queue.
    d. *Discover Azure message queues - Training | Microsoft Learn
        i. Service Bus, Azure Queue Storage, messages
    e. AZ-204: Developing solutions for Microsoft Azure (microsoftlearning.github.io)
    f. MessageReader console
    g. MessagePublisher console
11. Module 11: Monitor and optimize Azure solutions
    a. Application Insights OpenTelemetry observability overview - Azure Monitor | Microsoft Learn (Azure Monitor)

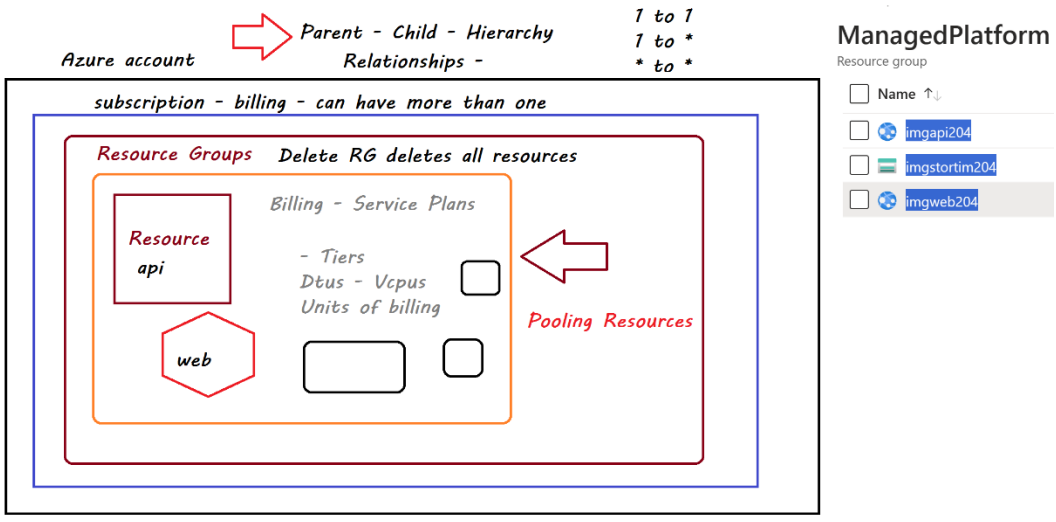
b. Azure Application Insights is an extensible Application Performance Management (APM) service provided by Azure Monitor.
c. It allows developers and DevOps teams to monitor the performance and usage of their live web applications, regardless of where they are hosted (cloud or on-premises).
d. It provides insights into application behavior, identifies performance bottlenecks, and helps diagnose errors.
e. Troubleshoot solutions by using Application Insights - Training | Microsoft Learn
f. Design to scale out - Azure Architecture Center | Microsoft Learn
g. Create an Application Insights resource in Azure that will be used to monitor and log application insight data for later review.
    i. smpapi
    ii. instrm
    iii. SimpleApi

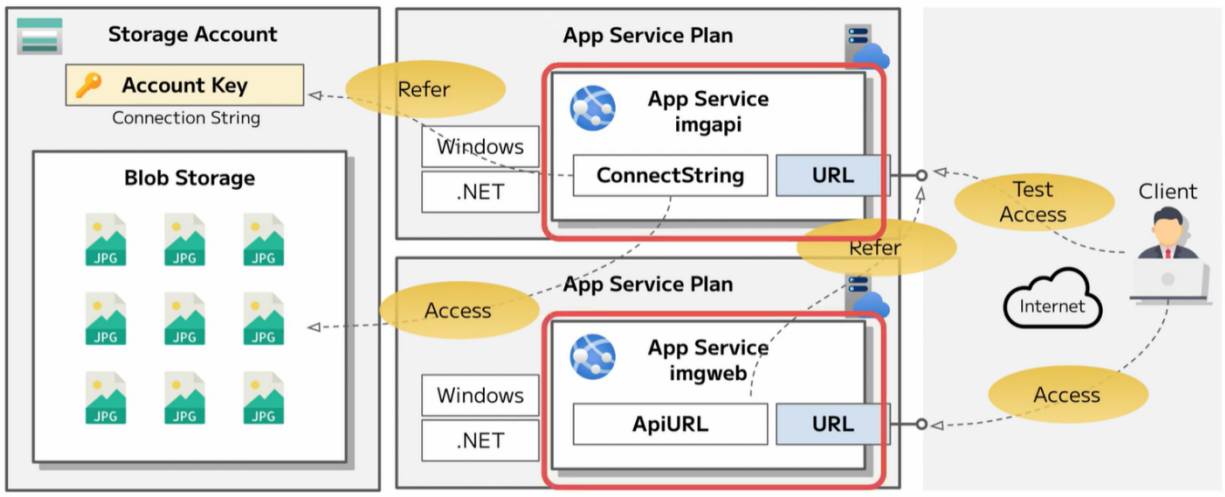
12. Module 12: Integrate caching and content delivery within solutions
    a. implement the Azure Content Delivery Network capabilities to provide a caching solution based on customer locations.
    b. The lab configures a storage account for image and video files, which are impacted the most by the latency issues.
13. End list

GitHub - MicrosoftLearning/AZ-204-DevelopingSolutionsforMicrosoftAzure: AZ-204: Developing solutions for Microsoft Azure

MS Learn Instructions for Labs: Just TOC no code.

AZ-204: Developing solutions for Microsoft Azure (microsoftlearning.github.io)





Module	Lab
1. Learning Path 01: Implement Azure App Service Web Apps	2. Lab 01: Build a web application on Azure platform as a service offerings
3. Learning Path 02: Implement Azure Functions	4. Lab 02: Implement task processing logic by using Azure Functions
5. Learning Path 03: Develop solutions that use blob storage	6. Lab 03: Retrieve Azure Storage resources and metadata by using the Azure Storage SDK for .NET
7. Learning Path 04: Develop solutions that use Cosmos DB storage	8. Lab 04: Construct a polyglot data solution
9. Learning Path 05: Implement containerized solutions	10. Lab 05: Deploy compute workloads by using images and containers
11. Learning Path 06: Implement user authentication and authorization	12. Lab 06: Authenticate by using OpenID Connect, MSAL, and .NET SDKs
13. Learning Path 07: Implement secure Azure solutions	14. Lab 07: Access resource secrets more securely across services

Module	Lab
15. Learning Path 08: Implement API Management	16. Lab 08: Create a multi-tier solution by using Azure services
17. Learning Path 09: Develop event-based solutions	18. Lab 09: Publish and subscribe to Event Grid events
19. Learning Path 10: Develop message-based solutions	20. Lab 10: Asynchronously process messages by using Azure Service Bus Queues
21. Learning Path 11: Troubleshoot solutions by using Application Insights	22. Lab 11: Monitor services that are deployed to Azure
23. Learning Path 12: Implement caching for solutions	24. Lab 12: Enhance a web application by using the Azure Content Delivery Network

PowerShell on Local Windows

1. Install Azure PowerShell on Windows | Microsoft Learn

```
$PSVersionTable.PSVersion
```

```
Major  Minor  Build  Revision
-----  -----  -----  --------
5      1      19041  3031
```

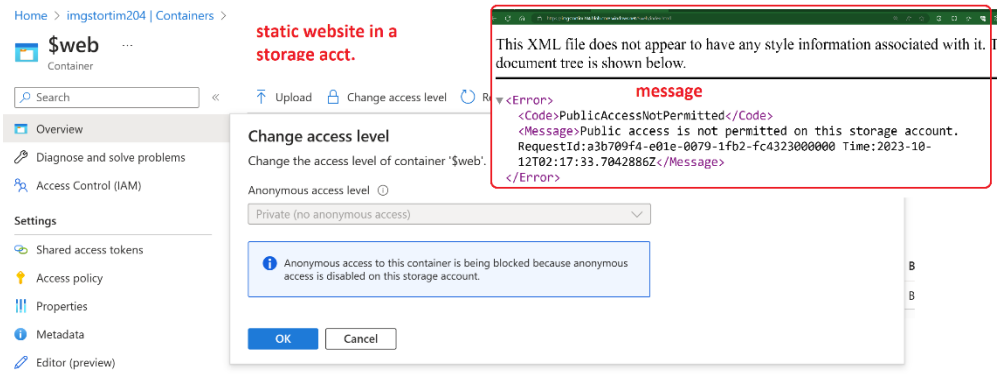
windows vs VS code

Same PC

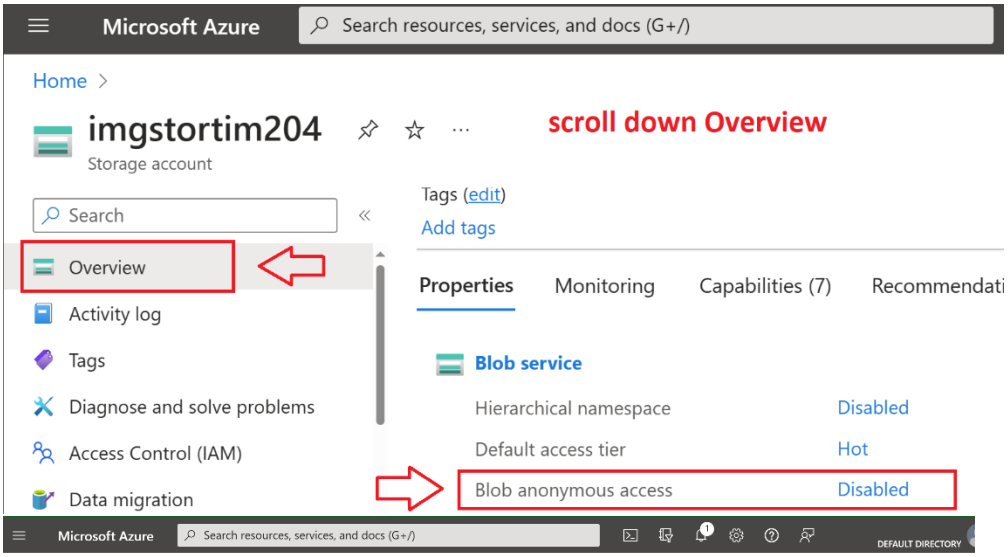
```
Major  Minor  Patch  PreReleaseLabel BuildLabel
-----  -----  -----  --------------- ----------
7      3      7
```

2.
3. Multiple versions of PowerShell – Inside VS Code vs. Win Terminal
4. Update-Module -Name Az -Force
5. HTTP response status codes - HTTP | MDN (mozilla.org)
6. 202: Accepted - indicates that server has received and understood the request, and that it has been accepted for processing
7. 500 : Internal Server error - indicates that the server encountered something it didn't expect and was unable to complete the request.
8. Static $web site in Storage account

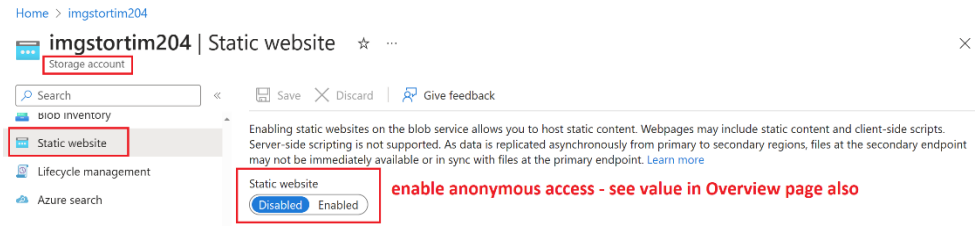
9.


static website in a storage acct.
message
This XML file does not appear to have any style information associated with it. The document tree is shown below.
<Error>
<Code>PublicAccessNotPermitted</Code>
<Message>Public access is not permitted on this storage account.
RequestId:a3b709f4-e01e-0079-1fb2-fc4323000000 Time:2023-10-12T02:17:33.7042886Z</Message>
</Error>
Home > imgstortim204 | Containers
$web
Container
Upload
Change access level
Overview
Diagnose and solve problems
Access Control (IAM)
Settings
Shared access tokens
Access policy
Properties
Metadata
Editor (preview)
Change access level
Change the access level of container '$web'.
Anonymous access level
Private (no anonymous access)
Anonymous access to this container is being blocked because anonymous access is disabled on this storage account.
OK
Cancel

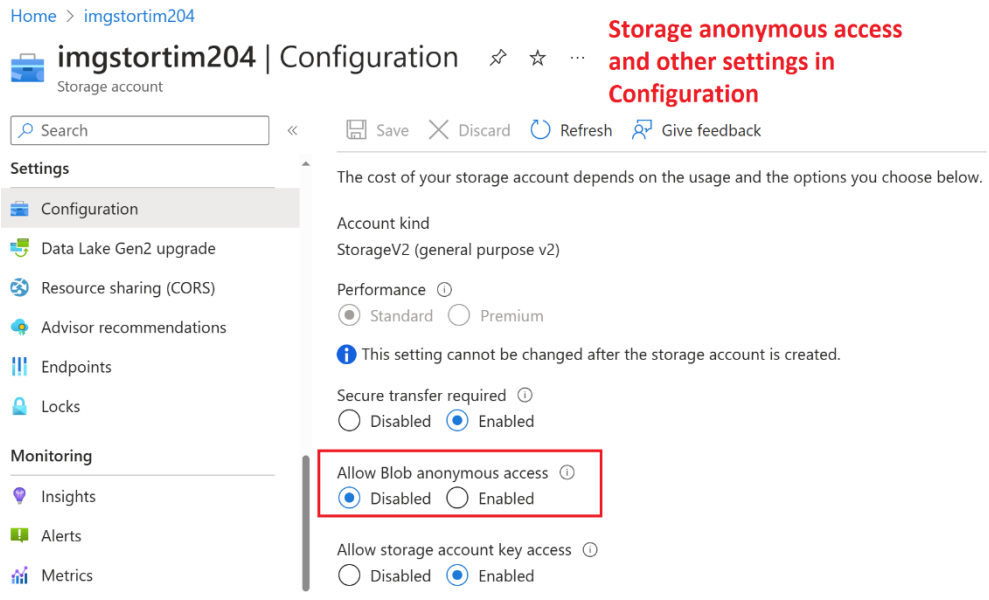
10.


Microsoft Azure
Search resources, services, and docs (G+/)
Home
imgstortim204
Storage account
scroll down Overview
Search
Overview
Activity log
Tags
Diagnose and solve problems
Access Control (IAM)
Data migration
Tags (edit)
Add tags
Properties
Monitoring
Capabilities (7)
Recommendati
Blob service
Hierarchical namespace
Disabled
Default access tier
Hot
Blob anonymous access
Disabled

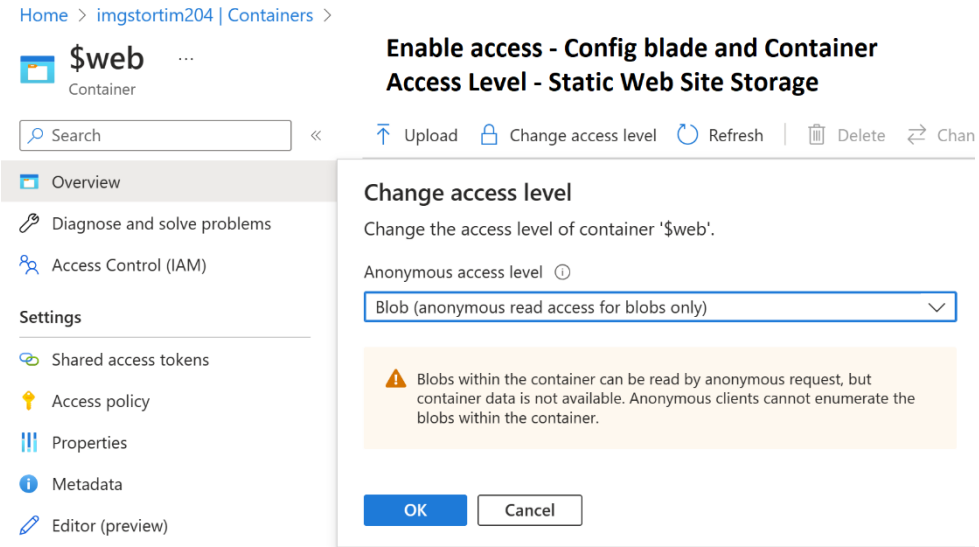
11.


Microsoft Azure
Search resources, services, and docs (G+/)
DEFAULT DIRECTORY
Home > imgstortim204
imgstortim204 | Static website
Storage account
Search
Save
Discard
Give feedback
Static website
Lifecycle management
Azure search
Enabling static websites on the blob service allows you to host static content. Webpages may include static content and client-side scripts. Server-side scripting is not supported. As data is replicated asynchronously from primary to secondary regions, files at the secondary endpoint may not be immediately available or in sync with files at the primary endpoint. Learn more
Static website
Disabled
Enabled
enable anonymous access - see value in Overview page also

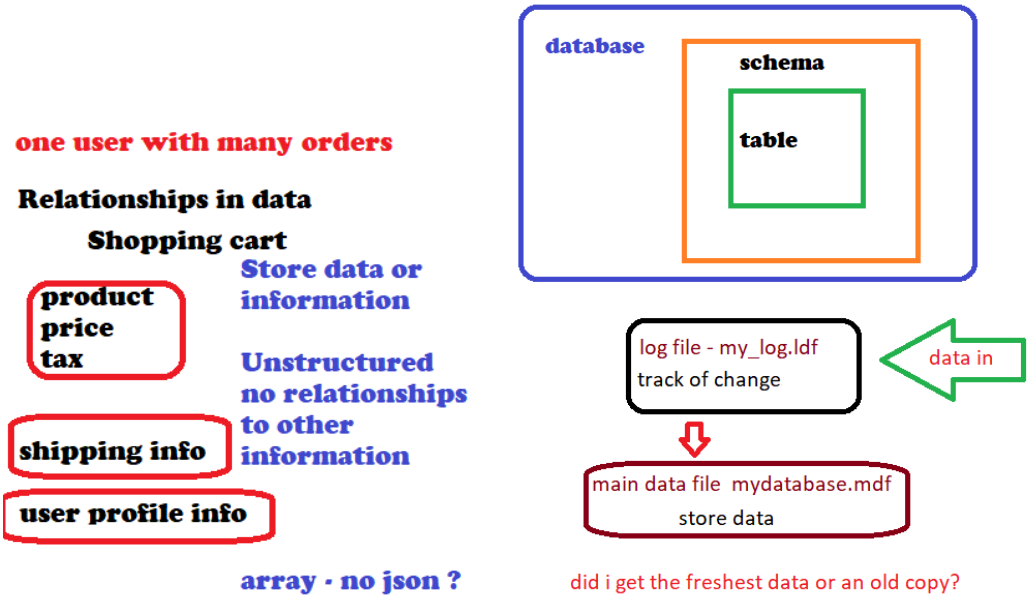
12.

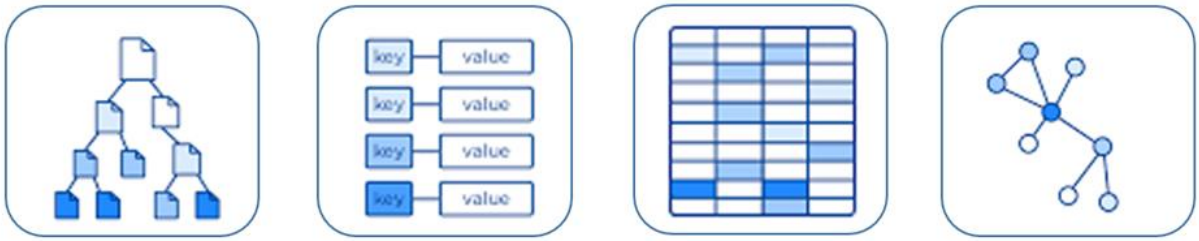

Home > imgstortim204
imgstortim204 | Configuration
Storage account
Storage anonymous access and other settings in Configuration
Search
Save
Discard
Refresh
Give feedback
Settings
Configuration
Data Lake Gen2 upgrade
Resource sharing (CORS)
Advisor recommendations
Endpoints
Locks
Monitoring
Insights
Alerts
Metrics
The cost of your storage account depends on the usage and the options you choose below.
Account kind
StorageV2 (general purpose v2)
Performance
Standard
Premium
This setting cannot be changed after the storage account is created.
Secure transfer required
Disabled
Enabled
Allow Blob anonymous access
Disabled
Enabled
Allow storage account key access
Disabled
Enabled

13.

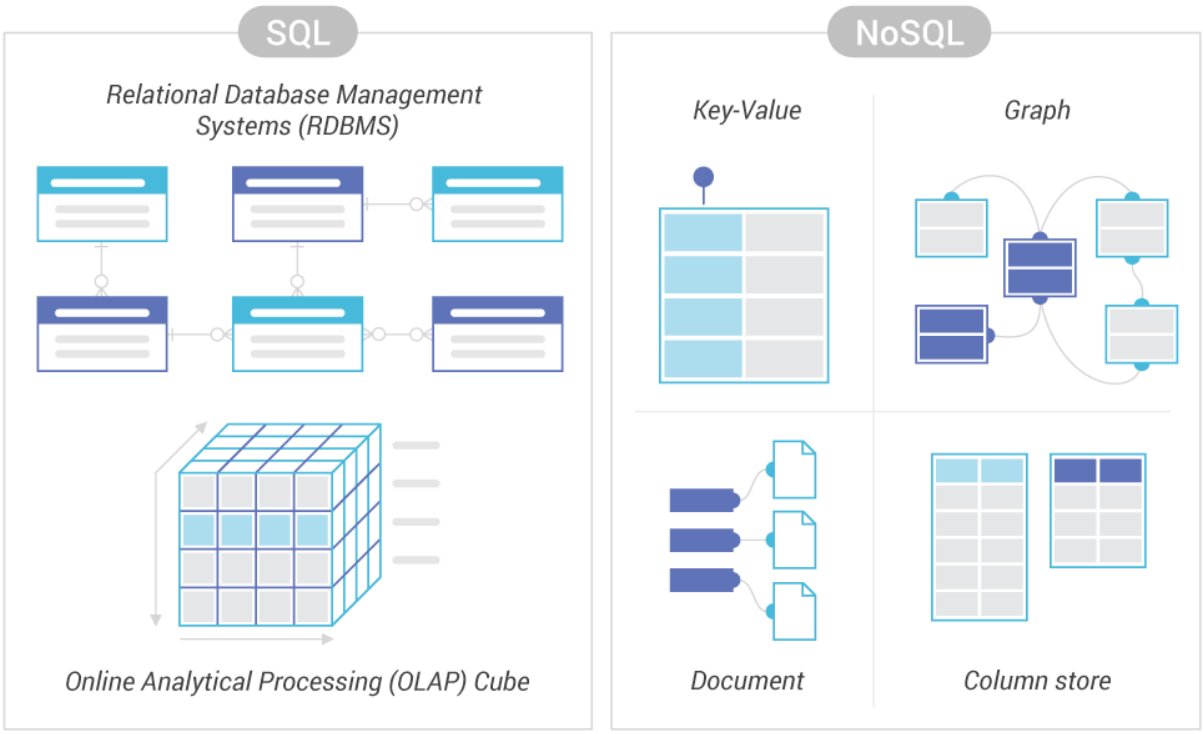

Home > imgstortim204 | Containers
$web
Container
Enable access - Config blade and Container Access Level - Static Web Site Storage
Search
Upload
Change access level
Refresh
Delete
Overview
Diagnose and solve problems
Access Control (IAM)
Settings
Shared access tokens
Access policy
Properties
Metadata
Editor (preview)
Change access level
Change the access level of container '$web'.
Anonymous access level
Blob (anonymous read access for blobs only)
Blobs within the container can be read by anonymous request, but container data is not available. Anonymous clients cannot enumerate the blobs within the container.
OK
Cancel

Data Movement - Storage

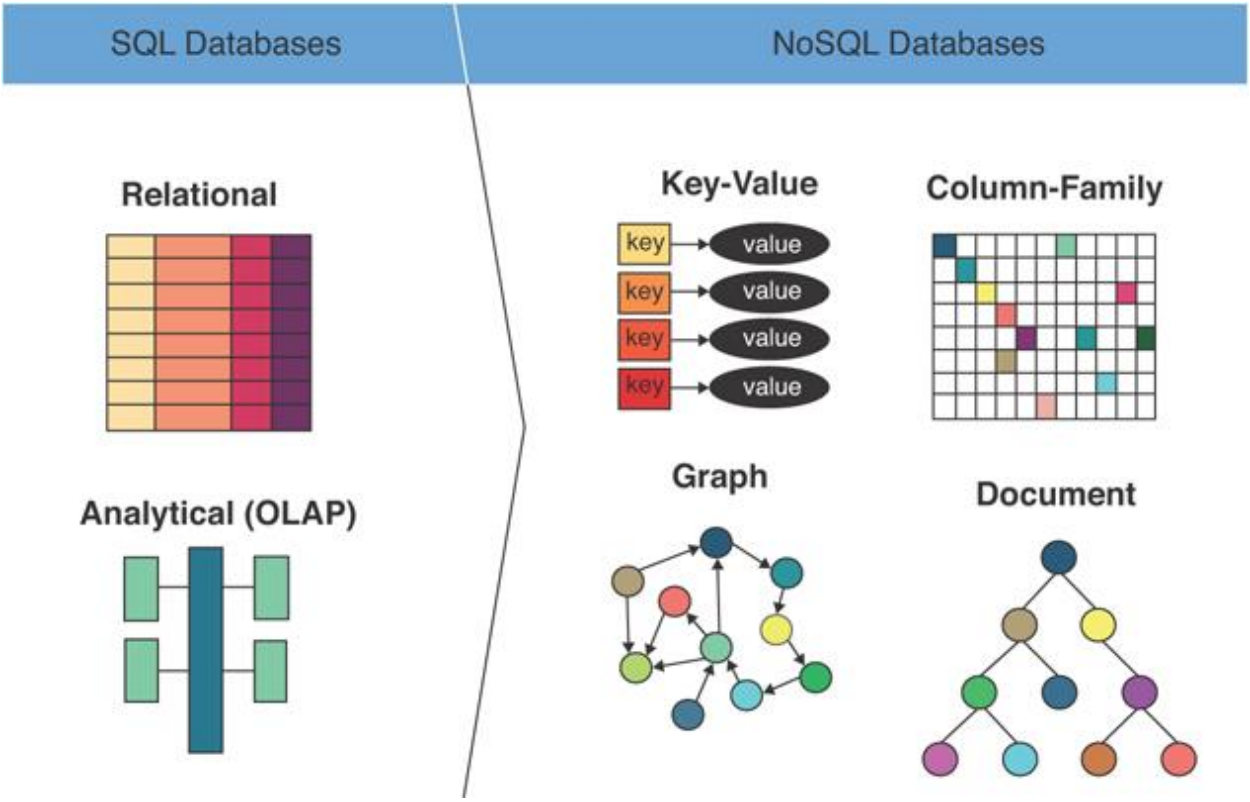
1. What are the boundaries – what makes up an "item" – Query it…
2. Question (report – data model – context – my object is a ?): How many bicycles are sold in Jan 2022 – Green in color – Size 26" wheel – boys – w/baskets
3. Data Relations – give data meaning: 1 to 1, 1 to many, many to many, many to 1
4. 

5. 

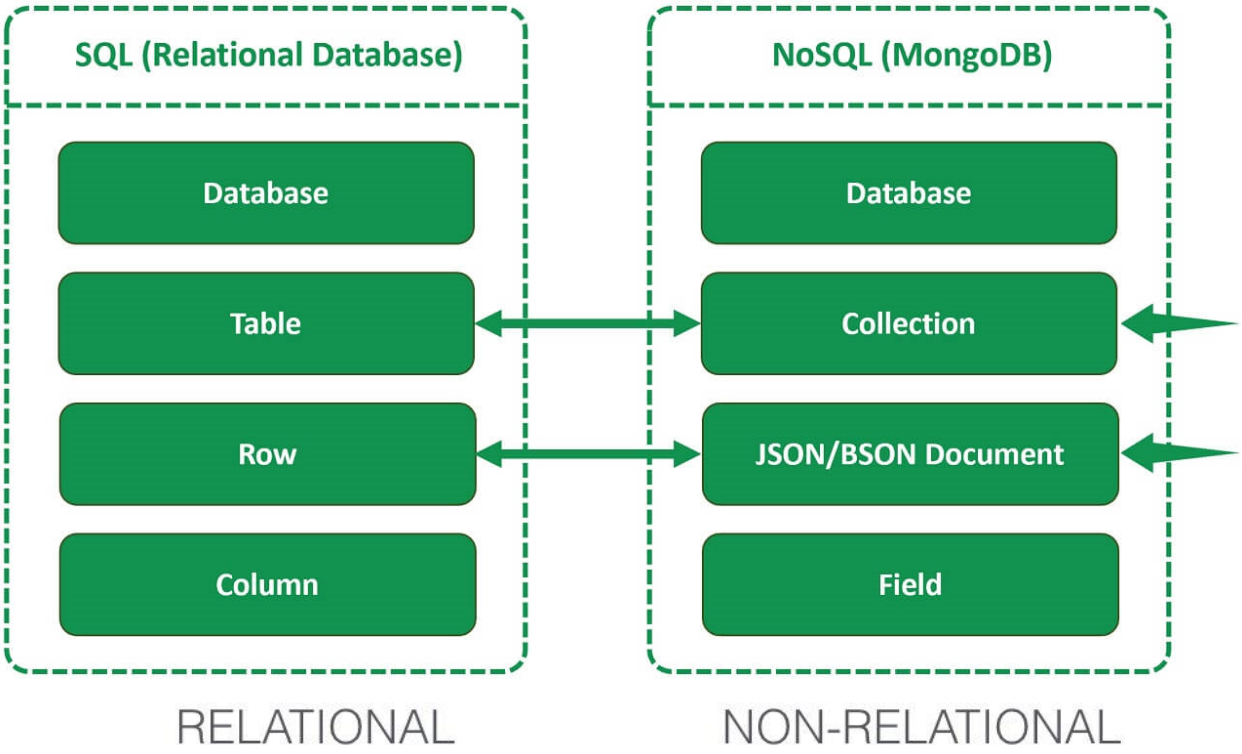
6.


SQL
Relational Database Management Systems (RDBMS)
Online Analytical Processing (OLAP) Cube
NoSQL
Key-Value
Graph
Document
Column store

7.


SQL Databases
NoSQL Databases
Relational
Analytical (OLAP)
Key-Value
key
value
Column-Family
Graph
Document

8.


SQL (Relational Database)
Database
Table
Row
Column
NoSQL (MongoDB)
Database
Collection
JSON/BSON Document
Field
RELATIONAL
NON-RELATIONAL

9.


Posts (id, Title)
1
Title
Comments
01
1
Comment 1
02
1
Comment 2

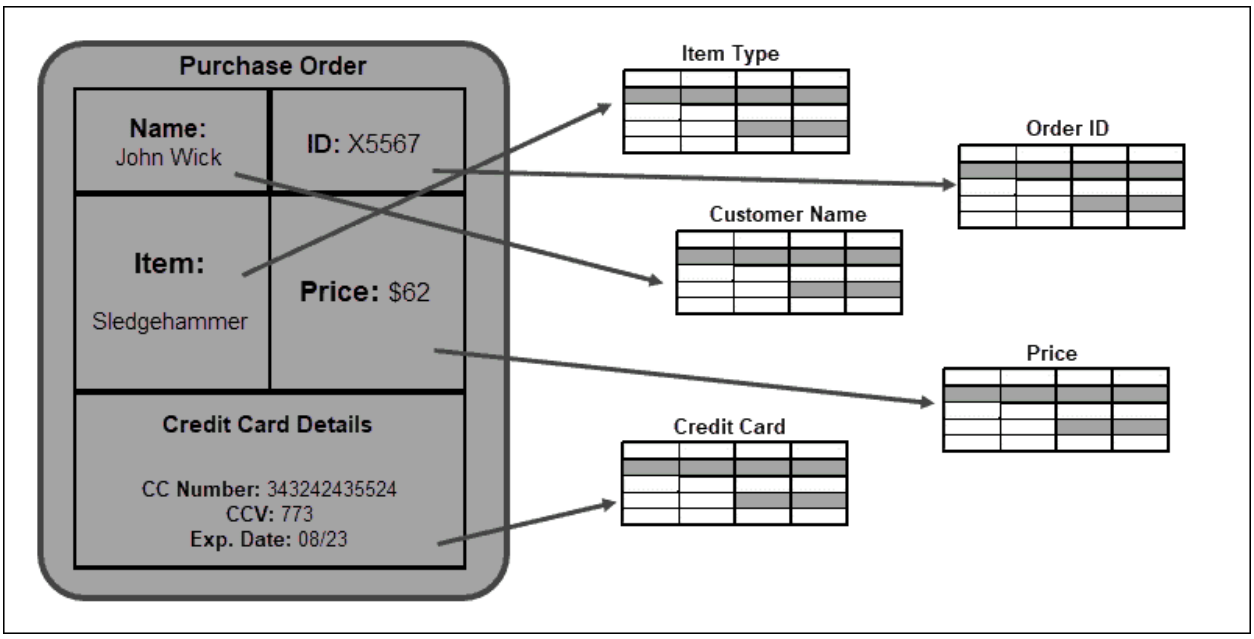

Posts (id, Title, Comments / Image)
1
Title
Comment 1
Comment 2
Comment 3
2
Title 2
Image

10.

11.

12.

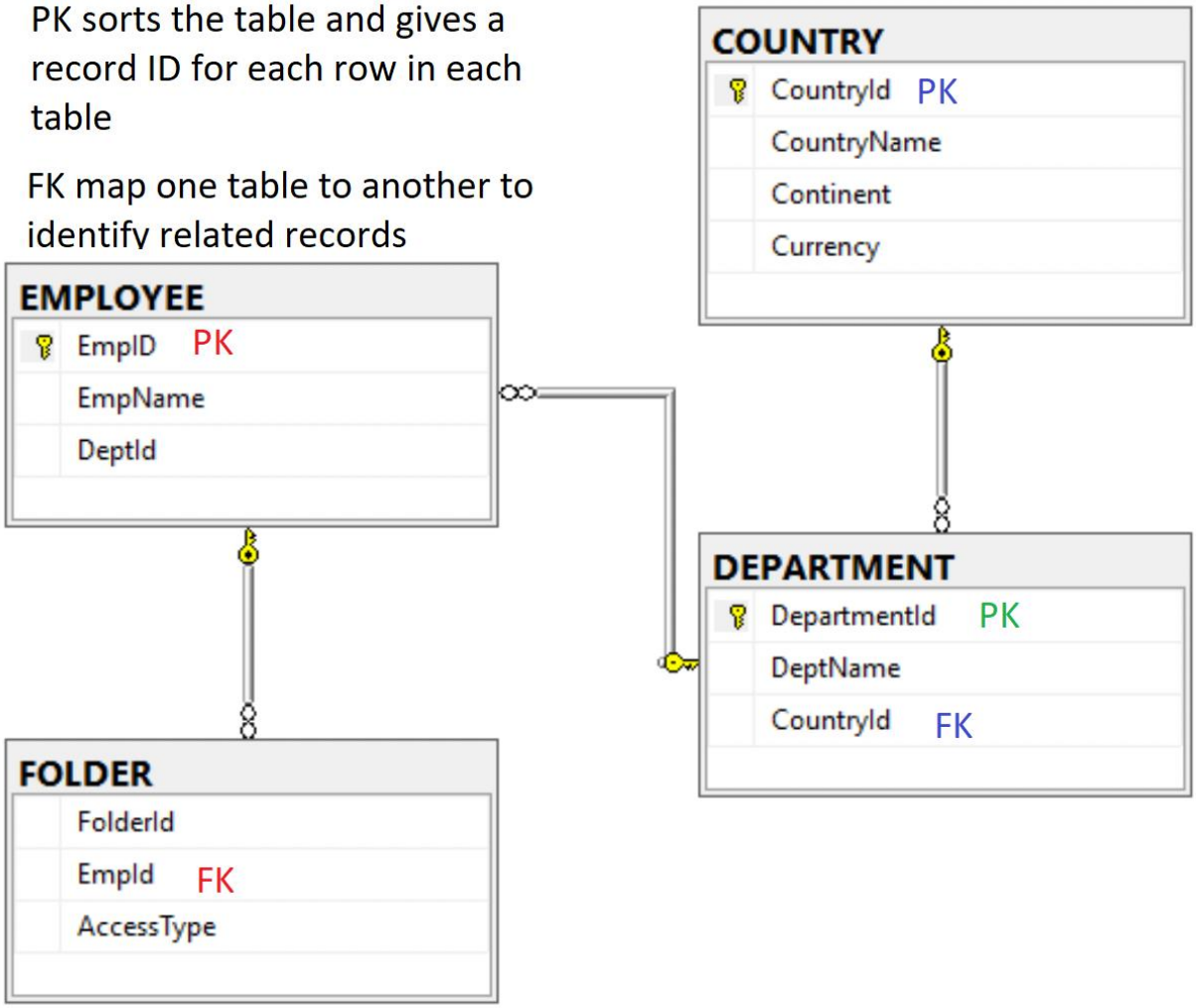
13.

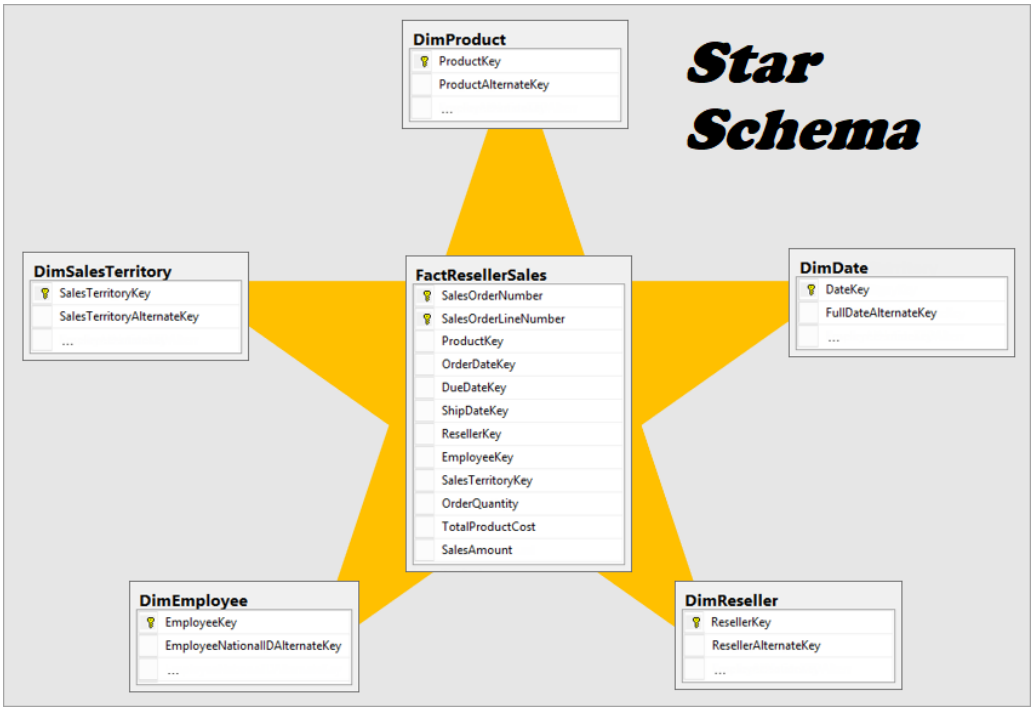
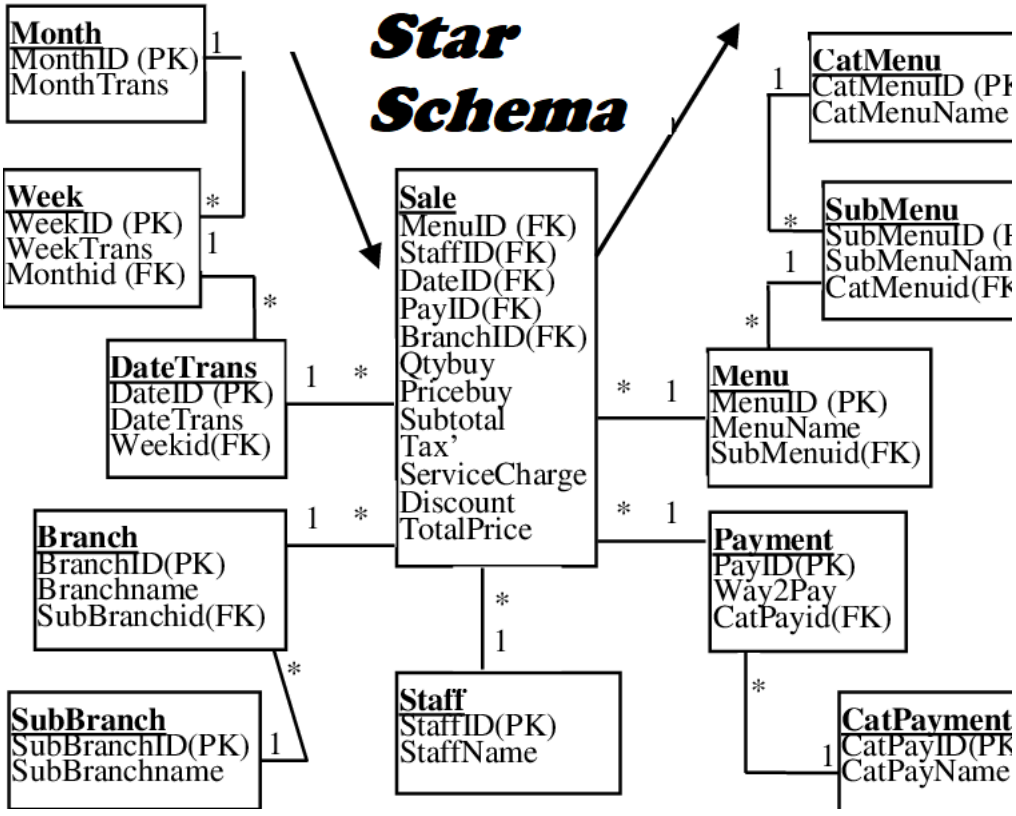


14.

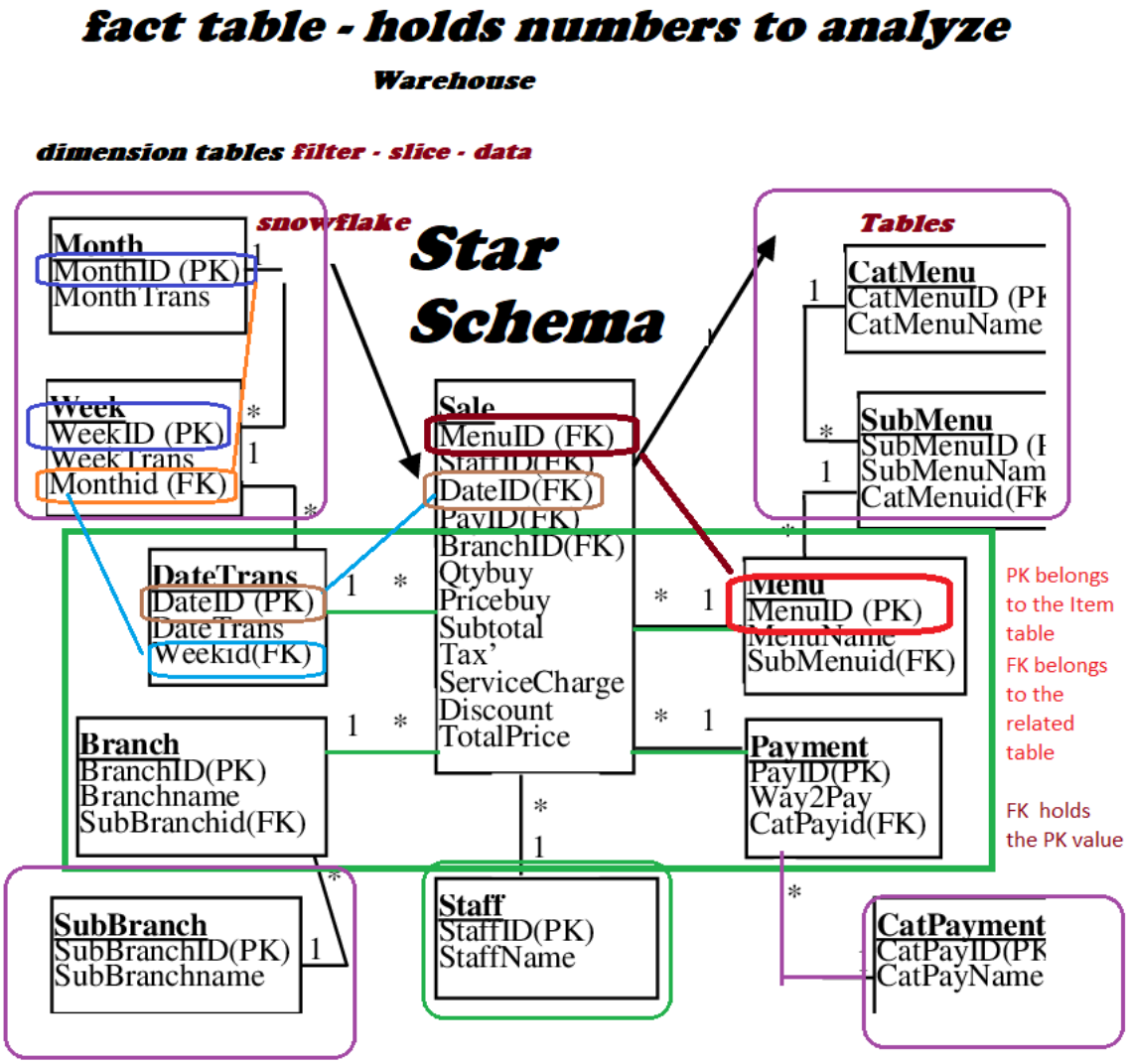
PK sorts the table and gives a record ID for each row in each table

FK map one table to another to identify related records



15. 
Star Schema
DimProduct
ProductKey
ProductAlternateKey
...
DimSalesTerritory
SalesTerritoryKey
SalesTerritoryAlternateKey
...
FactResellerSales
SalesOrderNumber
SalesOrderLineNumber
ProductKey
OrderDateKey
DueDateKey
ShipDateKey
ResellerKey
EmployeeKey
SalesTerritoryKey
OrderQuantity
TotalProductCost
SalesAmount
DimDate
DateKey
FullDateAlternateKey
...
DimEmployee
EmployeeKey
EmployeeNationalIDAlternateKey
...
DimReseller
ResellerKey
ResellerAlternateKey
...
16.
17.
18. 
Star Schema
Month
MonthID (PK)
MonthTrans
Week
WeekID (PK)
WeekTrans
Monthid (FK)
DateTrans
DateID (PK)
DateTrans
Weekid(FK)
Branch
BranchID(PK)
Branchname
SubBranchid(FK)
SubBranch
SubBranchID(PK)
SubBranchname
Sale
MenuID (FK)
StaffID(FK)
DateID(FK)
PayID(FK)
BranchID(FK)
Qtybuy
Pricebuy
Subtotal
Tax'
ServiceCharge
Discount
TotalPrice
Staff
StaffID(PK)
StaffName
CatMenu
CatMenuID (PK
CatMenuName
SubMenu
SubMenuID (P
SubMenuNam
CatMenuid(FK
Menu
MenuID (PK)
MenuName
SubMenuid(FK)
Payment
PayID(PK)
Way2Pay
CatPayid(FK)
CatPayment
CatPayID(PK
CatPayName

19.


fact table - holds numbers to analyze
Warehouse
dimension tables filter - slice - data
snowflake
Star Schema
Tables
Month
MonthID (PK)
MonthTrans
Week
WeekID (PK)
WeekTrans
Monthid (FK)
DateTrans
DateID (PK)
DateTrans
Weekid(FK)
Sale
MenuID (FK)
DateID(FK)
BranchID(FK)
Qtybuy
Pricebuy
Subtotal
Tax'
ServiceCharge
Discount
TotalPrice
CatMenu
CatMenuID (PK)
CatMenuName
SubMenu
SubMenuID (PK)
SubMenuName
CatMenuid(FK)
Menu
MenuID (PK)
MenuName
SubMenuid(FK)
Branch
BranchID(PK)
Branchname
SubBranchid(FK)
Payment
PayID(PK)
Way2Pay
CatPayid(FK)
SubBranch
SubBranchID(PK)
SubBranchname
Staff
StaffID(PK)
StaffName
CatPayment
CatPayID(PK)
CatPayName
PK belongs to the Item table
FK belongs to the related table
FK holds the PK value

20.

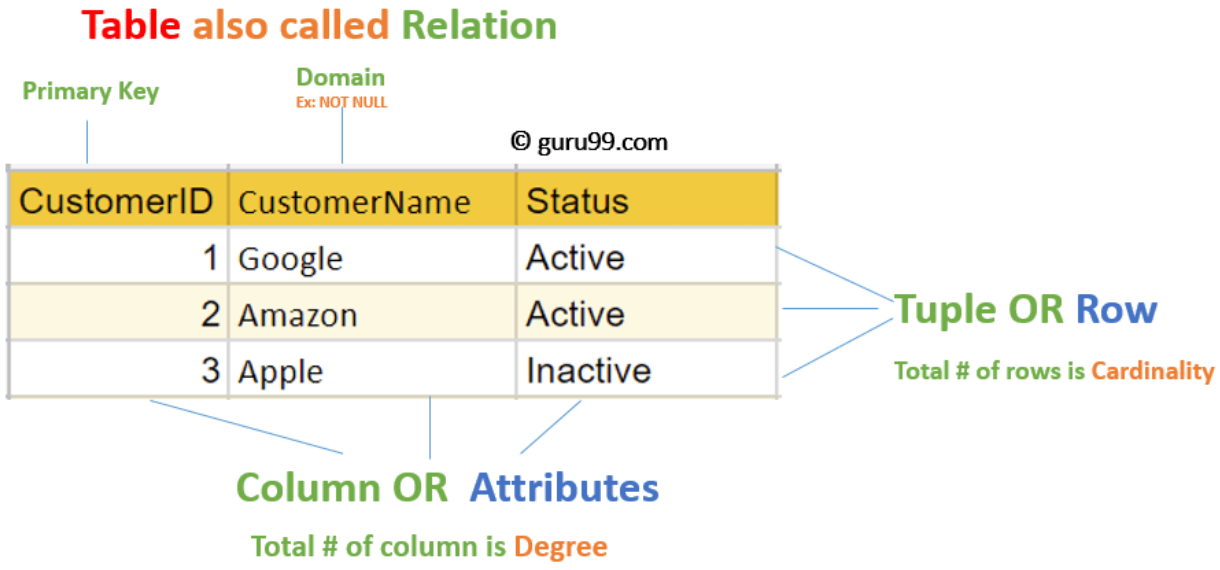
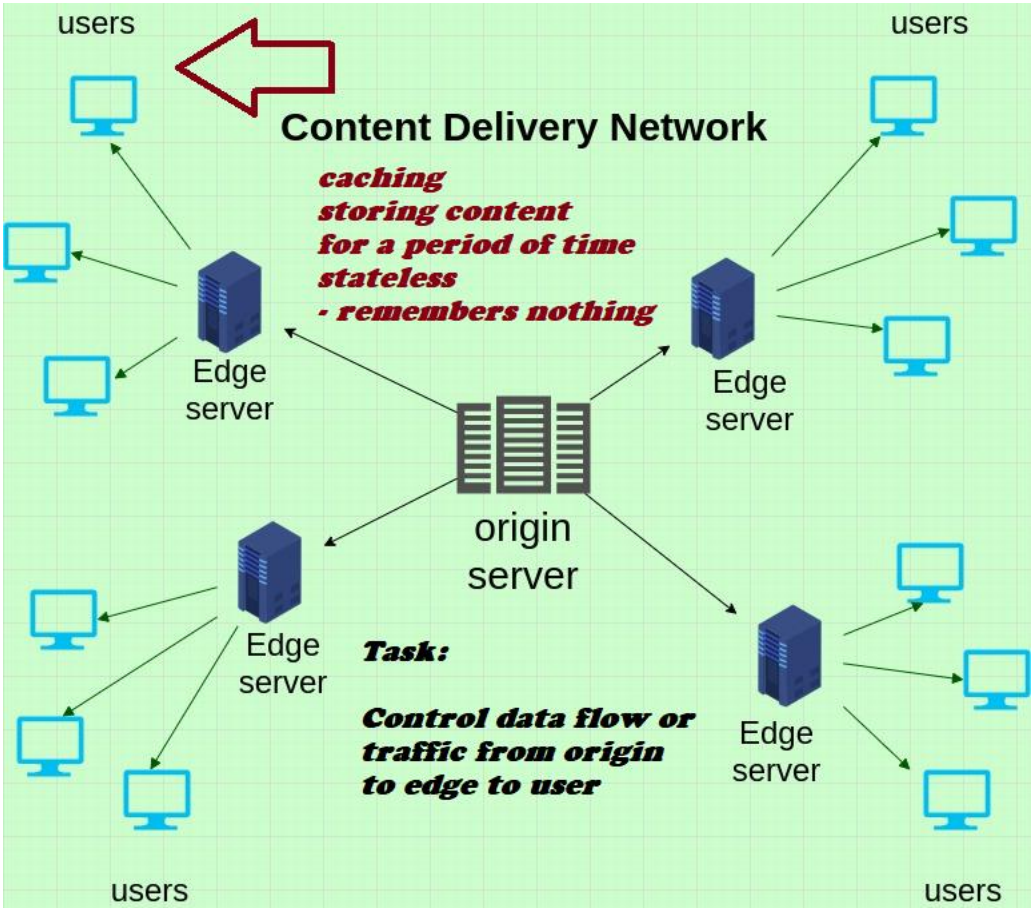
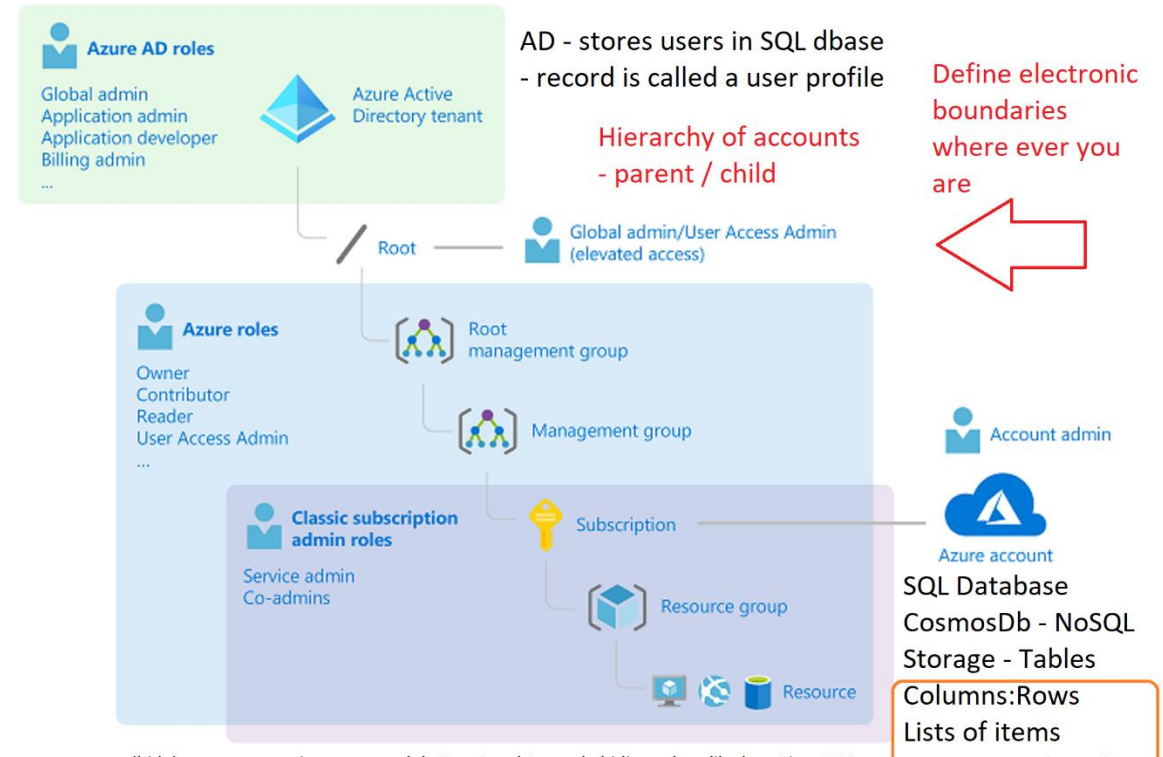


Table also called Relation
Primary Key
Domain
Ex: NOT NULL
© guru99.com
CustomerID
CustomerName
Status
1
Google
Active
2
Amazon
Active
3
Apple
Inactive
Tuple OR Row
Total # of rows is Cardinality
Column OR Attributes
Total # of column is Degree

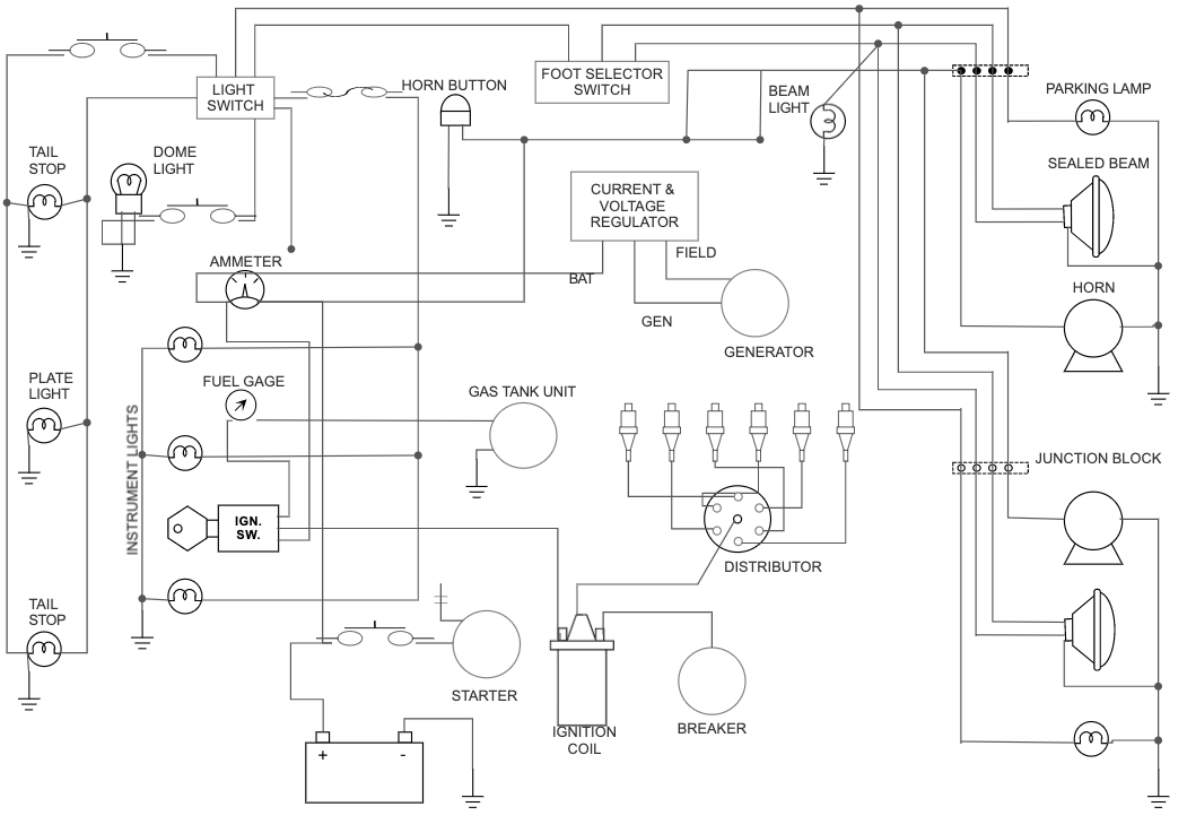
21.



22.



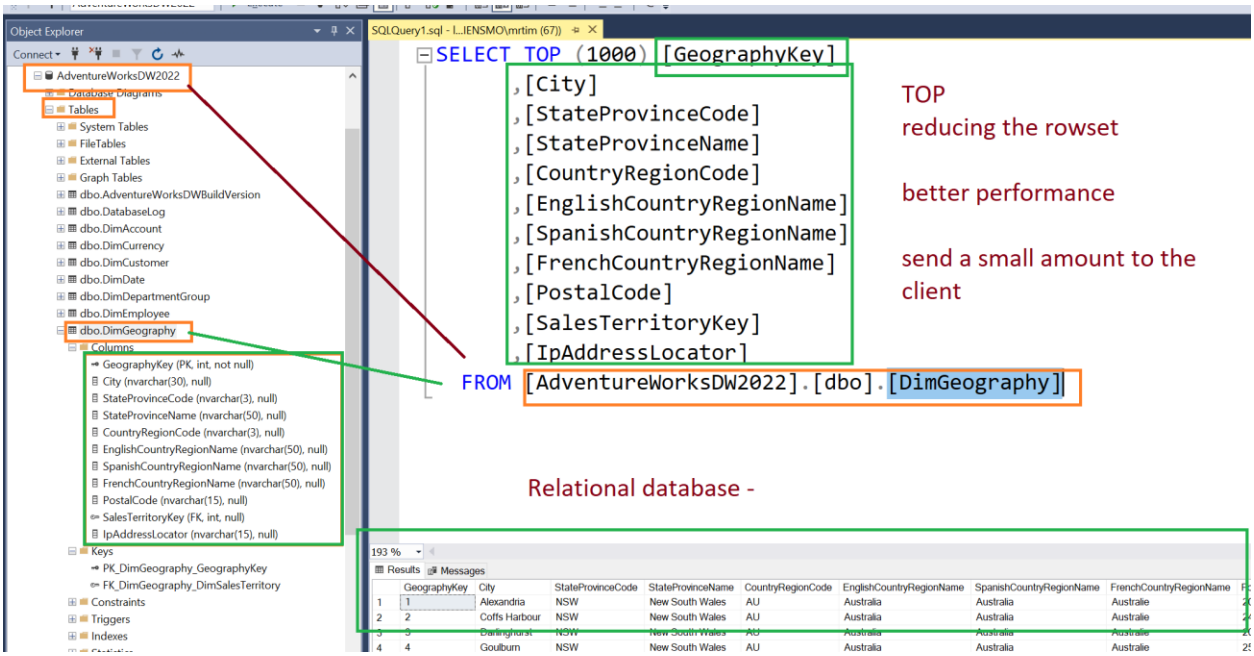
we can encrypt (hide) content - restrict access and do Row Level Control - hiding values like last 4 in a SSN



WIRING DIAGRAM AUTO ELECTRICAL WIRING DIAGRAM	DRAWN BY	CHECKED	DATE	SCALE	SHEET NO.

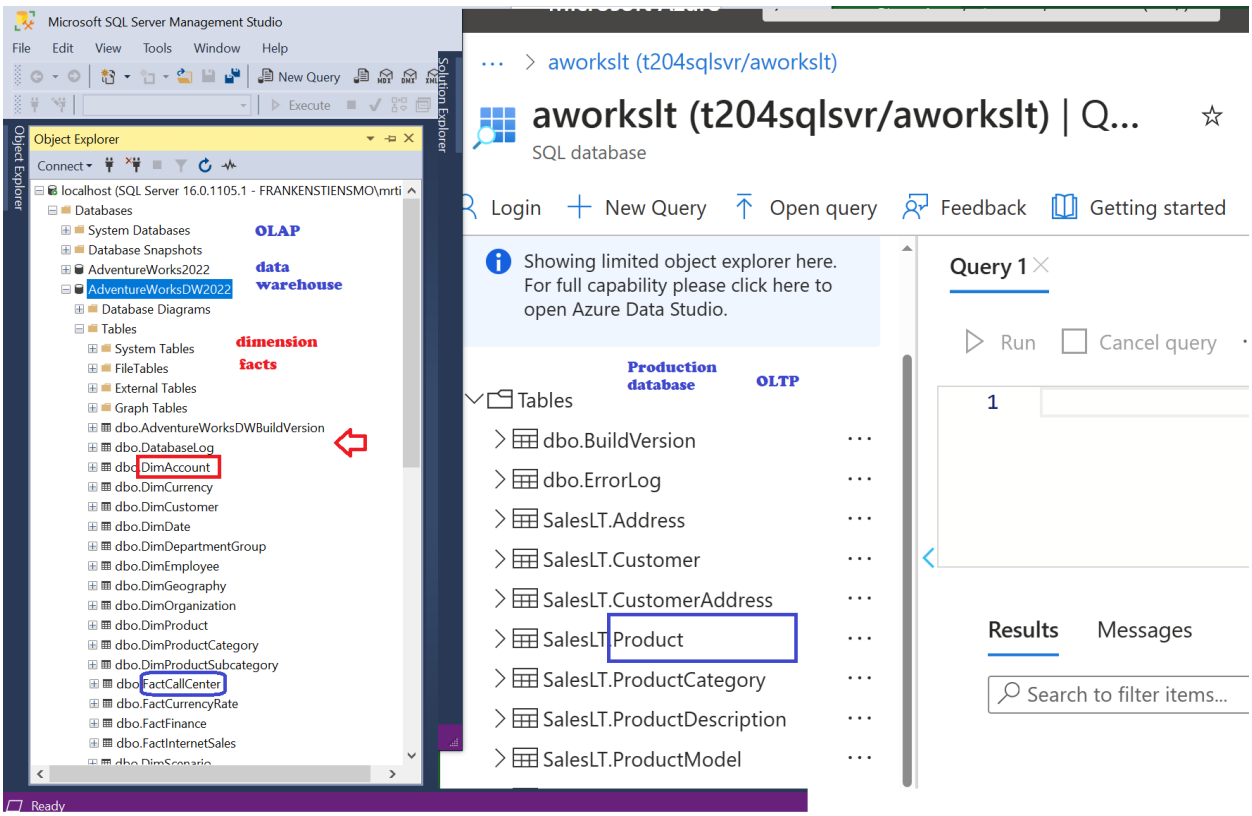
23.

Wiring Diagram



24.

25. SSMS Adventureworks database



26.
27. endlist

End to End App

Inventory of Apps:

1. Web App:
2. to upload existing web application files by using the Apache Kudu zip deployment option
3. Azure function app that echoes text that is entered and sent to the function by using HTTP POST commands.
4. BlobManager (03) created containers and managed blobs by using the Storage
5. C# code to query an Azure Cosmos DB collection (04)
6. EventPublisher – 09 - Event Grid events
7. MessagePublisher – 10 - Service Bus queue
8. Application Insights – 11 resource in Azure that will be used to monitor and log application insight data for later review.
9. Azure Content Delivery Network – 12 - Configure Content Delivery Network and endpoints

Commands Code Speak Cross Reference

Sure TD! Here's a quick comparison of **.NET CLI**, **Azure CLI**, and **PowerShell**—focused on their purpose, usage, syntax, and strengths:

Feature / Aspect	.NET CLI (dotnet)	Azure CLI (az)	PowerShell (pwsh)
🔧 **Primary Purpose**	Manage .NET apps/projects (build/run)	Manage Azure resources	Automate/admin Windows, Azure, cross-platform
🧠 **Focus Area**	Dev toolchain (C#, F#, VB.NET, etc.)	Cloud operations (infrastructure/devops)	Scripting & system automation
📦 **Install Package**	.NET SDK	Azure CLI	PowerShell Core or Windows PowerShell
💻 **Common Use**	dotnet build, dotnet run, dotnet new	az login, az vm create, az group list	Get-Process, Get-Service, New-AzVM
💬 **Command Style**	Verb-based + app-oriented (dotnet new)	REST-style + JSON-focused (az vm show)	Verb-Noun (Get-Item, Set-Content)
🧱 **Extensibility**	Templates, custom tools via SDKs	Extensions (e.g. az aks-preview)	Modules (Az, PSReadLine, etc.)
⚙️ **Cross-platform?**	Yes (Windows, macOS, Linux)	Yes	Yes (Core); WinPS is Windows-only
📄 **Output Format**	Console output (some JSON)	JSON (default), table, TSV, YAML	Objects (can pipe/format with Format-Table)
🔁 **Pipelining Support**	Basic piping (`	`)	Basic (text/json piping)
🧪 **Scripting Use**	Minimal scripting—mostly dev-focused	Moderate scripting (bash/zsh-friendly)	Rich scripting with error handling & logic

---

Summary

- ✅ **Use dotnet**: when you're working with .NET projects (build, run, publish, test).

- ✅ **Use az**: for Azure resource provisioning, automation, and devops tasks.
- ✅ **Use pwsh**: for admin tasks, scripting logic, and full automation—especially on Windows or with the Az module for Azure.

Great question — this is a key point in understanding modern software architecture. While **they all represent "code packaged for reuse"**, the terms *library*, *dependency*, *artifact*, *using reference*, and *SDK* reflect **differences in scope, purpose, or context**. Here's a breakdown:

---

1. Library

- **What it is:** A collection of reusable code (functions, classes, etc.).
- **Purpose:** Solve a specific problem or provide functionality (e.g., JSON parsing).
- **Form:** Usually a .dll, .jar, .so, or .py file.
- **Example:** Newtonsoft.Json.dll, requests in Python.

📌 **Think:** *Reusable functional unit*

---

2. Dependency

- **What it is:** Any code (typically a library) that your project depends on to compile or run.
- **Purpose:** Tracks what your code needs.
- **Form:** Could be a library, tool, or even an SDK.
- **Example:** In a .csproj: <PackageReference Include="Newtonsoft.Json" Version="13.0.1" />

📌 **Think:** *"What my code needs to work"*

---

3. Artifact

- **What it is:** A **built** output of a project (binary, package, image).
- **Purpose:** Deployed or stored product (not just source code).
- **Form:** .dll, .jar, .nupkg, .zip, .docker, etc.
- **Example:** MyApp-1.0.0.nupkg built from a .csproj

📌 **Think:** *"Built deliverable"*

---

4. Using Reference / Import

- **What it is:** The **code-level link** to use a dependency/library.
- **Purpose:** Tells the compiler you want to use code from somewhere else.
- **Form:** C# using Newtonsoft.Json;, Java import java.util.List;
- **Example:** Brings names into scope for code usage.

📌 **Think:** *"Code link to a library/dependency"*

🧰 5. SDK (Software Development Kit)

- **What it is:** A **complete toolkit** for building applications in a platform (includes libraries, tools, compilers, docs).
- **Purpose:** Not just a library — it's a development environment.
- **Form:** A package of CLI tools, libraries, templates, etc.
- **Example:** .NET SDK (dotnet CLI + runtime + libraries), Android SDK.

📌 **Think:** *"Everything needed to build for a platform"*

🎯 Summary Table

Term	Scope	Contains	Example	Analogy
Library	Functionality	Reusable code	Newtonsoft.Json.dll	A tool in a toolbox
Dependency	Contextual	Any required code	Added via nuget, pip, etc.	A recipe ingredient
Artifact	Output	Built binary/package	.nupkg, .jar, .dll	A baked cake
Reference	Code-level link	Code usage hook	using, import, etc.	Reading the label on a toolbox
SDK	Full toolkit	Libraries + tools	.NET SDK, Java SDK	A full kitchen with recipes

---

Hey TD — great question. Let's compare **curl**, **Azure CLI**, and other "scripting language"-like syntaxes in terms of purpose, style, and complexity. Here's the breakdown:

🛠️ 1. curl: The low-level HTTP tool

- **What it is**: A command-line tool to transfer data using URLs (mainly HTTP/S).
- **What it does**: Makes raw HTTP requests. You manually specify headers, methods, and bodies.
- **Strengths**: Universal, works anywhere, not tied to any platform.
- **Downsides**: Very verbose and error-prone for complex operations.

curl is like speaking directly to an API in Morse code.

```
curl -X POST https://example.com/api \
  -H "Authorization: Bearer TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"name":"TD"}'
```

🛠️ 2. Azure CLI: Task-focused abstraction

- **What it is**: A domain-specific command-line interface for managing Azure resources.
- **What it does**: Wraps REST calls to Azure APIs behind cleaner commands.
- **Strengths**: Intuitive for Azure tasks (VM creation, role assignment, etc.). Autocompletion and resource name inference.
- **Downsides**: Only works in Azure, limited control over raw HTTP.

Azure CLI is like a smart assistant that knows how Azure works and fills in a lot of blanks for you.

```
az vm create \
  --resource-group MyGroup \
  --name MyVM \
  --image UbuntuLTS \
  --admin-username azureuser \
  --generate-ssh-keys
```

🔁 Related syntaxes (comparison table)

Tool	Purpose	Abstraction Level	Scripting Style	Platform Specific	Ease of Use
curl	Raw HTTP/REST	Low	Procedural/manual	No	⚠ Low
az	Azure resource management	Medium-High	Declarative-like	✅ Yes (Azure)	✅ High
PowerShell	General automation + Azure	Medium	Verb-Noun (e.g., Get-AzVM)	✅ Yes (Windows/Azure)	High
Bash	General scripting	Low	Procedural	❌ No	⚠ Medium
Terraform	Infra as code (IAC)	High	Declarative HCL	Mostly Azure-ready	✅ High
ARM/Bicep	Infra as code (IAC)	High	Declarative JSON (ARM), clean (Bicep)	✅ Yes (Azure)	Varies

---

🧠 **TL;DR**

- curl is **lowest-level**: closest to the wire, least friendly.
- Azure CLI is **task-focused and friendly** for Azure, sits one level above REST.
- PowerShell adds **structure and composability** for automation beyond just cloud.
- Scripting tools like bash or curl are more **generic but primitive**.
- Azure CLI isn't "silly"—it's quite **efficient for infrastructure tasks**—but curl is often the **blunt instrument** when no SDK or CLI exists.

Want a practical example comparing the same Azure task with curl vs Azure CLI?

Yes — the curl -X GET -i http://localhost:7071/api/GetSettingInfo command is just a simple HTTP GET request to your local Azure Function endpoint. You *can* do the same with **Azure CLI** or **PowerShell**, although both are overkill for a simple local GET.

Here's how you'd translate it:

✅ PowerShell Equivalent

Invoke-RestMethod -Uri "http://localhost:7071/api/GetSettingInfo" -Method GET

Or if you want to see headers like -i in curl:

Invoke-WebRequest -Uri "http://localhost:7071/api/GetSettingInfo" -Method GET

✅ Azure CLI Equivalent

Technically, Azure CLI isn't designed to hit arbitrary HTTP endpoints — it's built for Azure resource management. But you *can* use the az rest command to hit REST APIs, even custom ones like your local Function:

az rest --method get --url "http://localhost:7071/api/GetSettingInfo"

This works, but is a bit unnatural — Azure CLI assumes Azure endpoints and authentication, so it's a stretch here.

💡 When to Use What

Task	Best Tool
Quick local test	✅ curl or Invoke-RestMethod
Azure REST APIs (authenticated)	✅ az rest or Invoke-AzRestMethod
Scripted integration with Azure Functions	✅ PowerShell / Invoke-RestMethod

---

Let me know if you want a version that **adds headers or a JSON body** — happy to extend!

Great follow-up — here's why **PowerShell or Azure CLI can feel like overkill** for something as simple as a local GET:

🧱 1. Verbosity / Complexity

curl is designed to do one thing well: make HTTP requests with minimal ceremony.

- curl:
- curl http://localhost:7071/api/GetSettingInfo
- PowerShell:
- Invoke-RestMethod -Uri "http://localhost:7071/api/GetSettingInfo" -Method GET
- Azure CLI:
- az rest --method get --url "http://localhost:7071/api/GetSettingInfo"

PowerShell and Azure CLI require extra syntax, keywords, or parameters for what is conceptually a one-liner HTTP request.

🧰 2. Purpose-built Tools

- curl is **built for raw HTTP** — it doesn't care if the target is Azure, localhost, or a microwave.
- PowerShell and az are **meant for automation, scripting, or Azure interaction**, not just poking endpoints.

So if all you're doing is testing a **single HTTP GET on a localhost dev server**, the **simpler tool wins**.

🧠 3. Fewer Dependencies

- curl is available by default on **Linux, macOS, Windows 10+**, WSL, etc.
- PowerShell is mostly on Windows, or needs to be installed elsewhere.
- Azure CLI is even more niche: you have to install/configure it, and it assumes you want Azure auth, context, subscriptions, etc.

🔍 When PowerShell or CLI *do* make sense:

- You're scripting multiple calls or processing the response (ConvertFrom-Json, error handling).
- You need fine control (headers, body, auth).
- You want to integrate into deployment scripts (ARM, Terraform, etc.).
- You're already in the Azure CLI or PowerShell shell environment.

🧪 TL;DR:

Use curl for:

- Quick manual API hits
- Testing localhost
- Minimal output or temporary use

Use PowerShell/Azure CLI for:

- Full scripting
- Structured automation
- When you're already in that ecosystem

Want me to show all three options with headers, JSON body, or auth tokens to really highlight the difference?

Great question — func init is used with the **Azure Functions Core Tools** (func) to **initialize a new Azure Functions project** locally.

✅ Expected Outcome of func init

When you run:

func init <folder-name> --worker-runtime <runtime> --target-framework <framework>

or simply:

func init

It will:

📁 Create a Project Structure

Depending on the language/runtime, func init sets up:

<your-project-folder>/

|

├── host.json # Global function app settings

├── local.settings.json # Local-only settings (e.g., connection strings)

├── .vscode/ # (Optional) Debug config for VS Code

└── .funcignore # Like .gitignore but for publishing/deploying

For C#, it may also create:

MyProject.csproj

For Node.js or Python, it sets up a basic structure without build files.

🧠 Context After Init

- You now have a **function app container** (a directory that can host one or more functions).
- You are ready to run func new to **add individual functions** (like HTTP triggers).
- It prepares for **local testing**, debugging, and eventual deployment to Azure.

⚙️ Typical Flags and Options

Option	Description
--worker-runtime	Language runtime (dotnet, node, python, etc.)
--target-framework	For .NET — specify net6.0, net8.0, etc.
--docker	Create Docker-related files
--csx	Use C# script-based model

Example:

func init MyApp --worker-runtime dotnet --target-framework net8.0

cd MyApp

func new --name MyHttpFunction --template "HTTP trigger"

🔍 TL;DR:

func init sets up the **shell of an Azure Functions app**, so you can:

- Add functions
- Run and test locally
- Deploy to Azure

Want a sample func init for Node.js, Python, or PowerShell?

Yes — in **Visual Studio Code**, there is a "Build" mechanism that is roughly equivalent to running dotnet build in the terminal — **but it depends on your setup** and project type.

✅ If You're Using a C#/.NET Project in VS Code

You can build your project using:

1. Command Palette

- Press Ctrl+Shift+P (or Cmd+Shift+P on Mac)
- Type: Tasks: Run Build Task
- Select the default build task (usually runs dotnet build)

VS Code uses **Tasks** under the hood to trigger builds.

2. Shortcut

- Press Ctrl+Shift+B
- This runs the default build task (again, dotnet build if it's configured that way)

3. Terminal (Manual)

- Open terminal (Ctrl+`)
- Run:
- dotnet build

This is the **same as what VS Code does**, unless you've customized your tasks.json.

⚙️ **Behind the Scenes: tasks.json**

If no build task is defined yet, VS Code will prompt to create it. A typical tasks.json might look like:

```
{
 "version": "2.0.0",
 "tasks": [
  {
   "label": "build",
   "command": "dotnet",
   "type": "process",
   "args": [ "build", "${workspaceFolder}/MyApp.csproj" ],
   "problemMatcher": "$msCompile"
  }
 ]
}
```

🧠 TL;DR

Method	Equivalent to dotnet build	Notes
Ctrl+Shift+B	✅ Yes	Fastest way to trigger build in VS Code
Tasks: Run Build	✅ Yes	Command Palette way
Terminal	✅ Yes	Manual but exact and visible
F5 (Debug)	🟡 Sometimes	Also builds, but mainly runs/debugs

Let me know if you're building **Azure Functions**, **Blazor**, or something else — there's some nuance for each.

Excellent question — yes, **build absolutely compiles your code** into new output files like .dll, .exe, .pdb, and more — but it also does **a few other important things** behind the scenes.

✅ Primary Purpose of dotnet build

✔️ **It compiles your source code into intermediate or deployable binaries**.

🔨 What dotnet build does specifically:

Step	Purpose
Compiles .cs files	Converts your C# code into IL (Intermediate Language)
Generates .dll	Produces assemblies (.dll for libraries, sometimes .exe for apps)
Generates .pdb	Produces debug symbols for IDEs to step through your code
Resolves dependencies	Pulls and links NuGet packages, references, etc.
Creates output folder	Usually /bin/Debug/net8.0/ or similar

Example:

/bin/Debug/net8.0/

├── MyApp.dll

├── MyApp.pdb

├── dependency1.dll

└── other-runtime-files

📁 Build vs. Publish vs. Run

Command	Does it compile?	Does it package for deployment?	Notes
dotnet build	✅ Yes	❌ No	Prepares for local dev/test/debug
dotnet run	✅ Yes (if needed)	❌ No	Builds + runs, good for local execution
dotnet publish	✅ Yes	✅ Yes	Produces deployable folder or single .exe

dotnet publish = dotnet build + copy dependencies + trim + bundle output

---

💡 TL;DR

- ✅ **build** = compiles your source to .dll (and maybe .exe) so it can be **run, tested, or debugged**.
- 🗜️ **publish** = prepares it for **production deployment**, like to Azure or a folder.
- 🧪 **run** = builds and executes, good for local testing but not meant for packaging.

Let me know if you want to break down the differences **for Azure Functions or Blazor**, where publish behaves slightly differently.

Great question — where ILogger messages show up depends on **where you're running your Azure Function App**: local dev environment or deployed to Azure.

🧪 1. Local (running with func start or in VS Code)

When running locally, ILogger output appears in the **terminal/console**.

✅ Options:

- If using:
- func start

You'll see lines like:

[2025-06-25T16:22:44.123Z] Executing 'MyFunction' (Reason='...', Id=...)

[2025-06-25T16:22:44.456Z] C# HTTP trigger function processed a request.

[2025-06-25T16:22:44.789Z] This is my custom ILogger log message

- If debugging in **Visual Studio Code**, ILogger output shows in the **Debug Console**.

☁️ 2. Deployed to Azure (in a live Function App)

✅ You can view logs in:

Where	How to Access	Notes
Azure Portal > Function App > Monitor	UI view of recent executions and logs	Great for quick inspection
Azure Portal > Log Stream	Live tail of logs	Works like a terminal stream
Application Insights	Kusto Query Language (KQL) queries for deep analysis	Only available if App Insights is enabled
Azure CLI / PowerShell	az functionapp log stream	Streams logs directly to terminal

Example (Azure CLI):

az functionapp log stream --name <FunctionAppName> --resource-group <ResourceGroup>

KQL example for App Insights:

traces

| where timestamp > ago(1h)

| where message contains "MyFunction"

| order by timestamp desc

Make sure you **enable Application Insights** when creating or configuring your Function App to get persistent, queryable logs.

🔧 Tips for Seeing Your ILogger Messages

- Use log.LogInformation("message") or higher (Warning, Error) — Trace might be filtered by default.
- Set "logging": { "logLevel": { "Default": "Information" } } in host.json to control what gets emitted.
- Log context-rich info (log.LogInformation("Request received with name: {name}", name))

🧠 **TL;DR**

Environment	Where to View ILogger Logs
Local dev	Terminal / Debug Console in VS Code
Azure	**Log Stream**, **Monitor tab**, **App Insights**, or az functionapp log stream

Let me know if you want a sample host.json or App Insights KQL query to search for custom log messages.

Hey! Great question — the term **API** is broad, and **REST API** is a specific type. Here's a breakdown:

🔷 **What is an API?**

API (Application Programming Interface) is a general term for any interface that allows two applications or systems to communicate. APIs define how requests for services should be made, what data is required, and what kind of responses to expect.

🧠 Think of an API as a menu in a restaurant — it tells you what you can order, and how the kitchen will respond.

🔷 **What is a REST API?**

REST (Representational State Transfer) is a style or set of rules for building **web APIs** over HTTP.

A **REST API** is:

- A **web-based API** using HTTP methods like GET, POST, PUT, DELETE.
- Based on resources (usually URLs like /users/123).
- **Stateless** (each request contains all needed information).
- Uses standard formats like **JSON or XML** for data exchange.

🔶 **So, How is a REST API Different from Other APIs?**

Feature	REST API	Other APIs (e.g., SOAP, gRPC, SDKs, Local APIs)
Protocol	HTTP/S only	Could be HTTP, TCP, gRPC, local IPC, etc.
Format	Typically JSON (or XML)	SOAP: XML, gRPC: Protocol Buffers, others vary
Architecture Style	Stateless, resource-based	Could be stateful or RPC-like

Feature	REST API	Other APIs (e.g., SOAP, gRPC, SDKs, Local APIs)
Standards	Loose rules, very flexible	SOAP: strict WSDL contract, gRPC: strict schema
Transport	Always web/HTTP	Not always — e.g., DLL APIs or OS-level system APIs
Tooling	Easily testable with tools like Postman	May require client code or specific libraries

---

🔷 **Summary**

- **REST API** = one type of **web API** built on specific architectural principles.
- **API** is a generic term that can refer to many other types — not all are web-based or follow REST.

Let me know if you want examples comparing REST to others like SOAP, gRPC, or SDK-based APIs!

Yes — the use of a **connection string**, **access key**, or **endpoint** is primarily about **how you connect to a specific resource**, **authenticate**, and **communicate** with it. But each has a distinct role in that process:

🔑 **1. Access Key – *Authentication***

- Proves your identity or that your client app has permission to access the resource.
- Similar to a **password**.
- Examples:
  - Azure Storage account key
  - API keys in headers
  - AWS access keys

🌐 **2. Endpoint – *Location***

- Tells your app where the resource lives (like a URL or IP address).
- It's the **address of the service** you're calling.
- Examples:
  - https://myapi.service.azure.com
  - https://storageaccount.blob.core.windows.net
  - Localhost endpoints during dev

🧬 3. Connection String – *Bundle of Details*

- A **packaged string** that includes the endpoint, access key, and other info (sometimes ports, database name, etc.).
- Makes it easier to connect without configuring each part separately.
- Examples:
  - SQL Server: Server=myServer;Database=myDB;User Id=admin;Password=pass;
  - Azure Blob: DefaultEndpointsProtocol=https;AccountName=myaccount;AccountKey=abc...;EndpointSuffix=core.windows.net

---

📌 Summary Table

Component	Purpose	Example Use
Access Key	Authenticate client	API key, Azure Storage key
Endpoint	Locate the resource	URL to API or service
Connection String	Bundle of all connection info	SQL DB, Storage Account, Redis, etc.

✅ Bottom Line

Yes, they're all about connecting to **resources**, but:

- **Access keys** prove **who** is connecting.
- **Endpoints** define **where** to connect.
- **Connection strings** are the **convenient wrapper** with everything needed to establish that connection.

Let me know if you want to break this down for a specific Azure service or other scenario!

VM scale-up vs. Cosmos DB scale-out, both in mechanism and architecture:

Hey TD – Happy Monday to you too! Great Azure question. Let's break it down by comparing **VM scale-up** vs. **Cosmos DB scale-out**, both in **mechanism** and **architecture**:

🔧 1. Scale-Up (Vertical Scaling) – Azure Virtual Machines

✅ What it means:

- Increase the **size of the VM** (e.g., from Standard D2s to D8s).
- More **CPU, RAM, network bandwidth, and storage** per instance.

⚙️ **How it's done (Mechanism):**

- **Stop the VM** (in most cases).
- Resize to a **larger SKU** via Azure Portal, CLI, or PowerShell.
- Restart the VM.

Note: This is typically **manual or semi-automated**, and downtime may occur.

📌 **Characteristics:**

- Easy to manage, but **limited by hardware availability**.
- **Not elastic**—can't automatically grow/shrink per load in the moment.
- Works well for monolithic workloads or legacy apps.

🚀 **2. Scale-Out (Horizontal Scaling) – Azure Cosmos DB**

✅ **What it means:**

- **Add more partitions** (or replicas) across multiple nodes.
- Increases **throughput and storage** linearly.

⚙️ **How it's done (Mechanism):**

- Cosmos DB uses **logical partitions** (based on a partition key).
- Under the hood, these map to **physical partitions** across servers.
- Cosmos DB **automatically adds** more physical partitions when:
    - Throughput (RU/s) is increased.
    - Storage exceeds physical partition limits (~50GB per partition).

This is fully managed and elastic—no downtime, and auto-scaled in serverless or provisioned modes.

📌 **Characteristics:**

- Designed for **planet-scale**, high-throughput apps.
- Highly available (multi-region), low-latency, globally distributed.
- Requires a good **partition key** choice to scale effectively.

🧠 **Summary Table:**

Feature	Azure VM (Scale-Up)	Cosmos DB (Scale-Out)
Scaling Type	Vertical	Horizontal
Mechanism	Resize VM to larger SKU	Add physical partitions automatically
Manual or Auto	Manual/semi-auto	Fully automated
Downtime Risk	Possible during resize	No downtime
Best For	Compute-heavy single-instance tasks	Large-scale, partitioned NoSQL workloads
Limitations	Upper bound based on hardware	Partitioning and key design matters

---

Let me know if you want visuals, Azure CLI examples, or how this ties into autoscaling policies or cost impacts!

Azure Tables – Table Storage – API

are Azure Cosmos DB API for Table stores data in key/value format & using Azure Table storage or a supported api - https://learn.microsoft.com/en-us/training/modules/explore-azure-cosmos-db/6-cosmos-db-supported-apis - intended to connect no sql stores together that have different vendors?

You're asking a very insightful and nuanced question — and you're absolutely on the right track.

Let's unpack it piece by piece:

✅ **1. Does the Azure Cosmos DB API for Table store data in key/value format?**

Yes, the Cosmos DB Table API **stores data in a key/value schema**, very much like **Azure Table Storage**.

- It uses:
  - **PartitionKey**
  - **RowKey**
  - Other columns as **property bags** (key/value properties).

This makes it compatible with the existing **Azure Table Storage SDKs**, especially for apps already using them.

✅ **2. Does it use Azure Table Storage under the hood?**

No, Cosmos DB API for Table does **not use Azure Table Storage under the hood**.

- Instead, it's **a completely different back-end** — powered by Cosmos DB's globally distributed, highly available, low-latency infrastructure.
- What Cosmos DB *does* is **emulate** the **Azure Table Storage protocol** so your existing code continues to work with:
    - **Higher performance**
    - **Global distribution**
    - **Automatic scaling**
    - **Multi-region writes**
    - **SLA-backed guarantees**

Think of it like: **same interface, better engine**.

🔁 3. Is it intended to connect NoSQL stores with different vendors?

Not exactly — **the API for Table is not a multi-vendor NoSQL gateway**. However, here's where your idea overlaps with Microsoft's vision:

Cosmos DB API	Target Compatibility / Emulation
Core (SQL)	Cosmos-native JSON document DB
MongoDB API	Emulates MongoDB wire protocol
Cassandra API	Compatible with Cassandra SDKs
Gremlin API	For graph-based queries (Gremlin TinkerPop)
Table API	Compatible with Azure Table Storage

So yes — Cosmos DB is **multi-model** and supports **multiple APIs to help apps migrate or interoperate across vendors**.

But Cosmos DB does **not act as a federated query layer** across different vendors (e.g., MongoDB + Cassandra together). Instead, it **emulates** each separately so you can **bring workloads to Cosmos DB** without rewriting apps.

🔑 Summary:

Concept	Cosmos DB Table API
Key/value store?	✅ Yes — PartitionKey/RowKey model

Concept	Cosmos DB Table API
Same as Azure Table Storage?	❌ No — it's Cosmos DB back-end, with a Table Storage-compatible API
Use case?	✅ Migrate apps from Azure Table Storage or build apps needing globally distributed key/value data
Multi-vendor NoSQL bridge?	❌ Not directly — each API supports a single vendor compatibility mode

---

Let me know if you'd like to see how **Table API compares to Core (SQL)** or **how SDK usage changes (or doesn't)** across Table, Mongo, and Cassandra in Cosmos DB!

How does Azure Table storage manage data if not key/value?

Excellent follow-up — this is one of those cases where terminology matters a lot. So here's the deal:

✅ **Azure Table Storage *does* manage data using a key/value-like structure, but it's technically a bit more structured than a raw key/value store.**

🧩 **So how does Azure Table Storage manage data?**

It uses a **schema-less, partitioned table model**:

Component	Description
Table	A collection of **entities** (like rows in a table).
Entity	A set of properties (like a row with flexible columns).
PartitionKey	Groups related entities — helps with **partitioning & scalability**.
RowKey	Unique within a partition — acts like the **primary key** within a partition.
Properties	Dynamic name/value pairs (like columns), with typed values (e.g., string, int, GUID).

This makes Azure Table Storage a **semi-structured NoSQL key/value store**, with **two keys** (PartitionKey + RowKey) instead of just one.

📦 **Why is it more than just a raw key/value store?**

Most key/value stores (like Redis or Amazon DynamoDB's basic form) just store:

key → value (usually binary or JSON blob)

But **Azure Table Storage** adds some structure:

(PartitionKey, RowKey) → { Property1: Value1, Property2: Value2, ... }

- So it's more like a **multi-attribute key/value store** or **wide column store**.
- It doesn't enforce schema — different entities in the same table can have **different properties**.

🔄 **How it differs from Cosmos DB Table API**

Feature	Azure Table Storage	Cosmos DB Table API
Backend Engine	Azure Storage system	Cosmos DB (globally distributed)
Consistency Options	Strong (region-bound)	Multiple models (eventual, session, etc.)
Indexing	Only PartitionKey + RowKey	All properties auto-indexed
Scale	Limited by storage account	Virtually unlimited, multi-region
Latency + SLA	No SLA for latency	<10ms read, <15ms write (SLA-backed)
SDK/API Support	Azure Storage SDK	Same SDK, plus Cosmos DB SDKs

---

✅ **In summary:**

Feature	Azure Table Storage
Data Model	Keyed by (PartitionKey, RowKey)
Value Format	Dynamic set of typed properties (schema-less)
Technically KV?	✅ Yes — but with composite keys and structured values
Use Cases	Lightweight NoSQL storage for logs, user prefs, etc.

Let me know if you want a quick diagram showing the data structure or a code sample to show how a record is stored/accessed!

So I Yaml as a Language? hence the (parens) arrow function to JSON or Curly Braced syntax?

Great question — you're connecting the dots between **data languages** and **syntax metaphors** like parens (), arrows →, and curly braces {}. Let's clear it all up:

✅ Is YAML a Language?

Yes — **YAML (YAML Ain't Markup Language)** is considered a **data serialization language**, not a programming language.

- It's used to represent **structured data** (like configs or documents), often as an alternative to **JSON** or **XML**.
- It's **human-readable**, using indentation instead of braces, and supports comments (#), complex structures, anchors/aliases, etc.

🧠 Comparing YAML, JSON, and (syntax metaphors)

Let's break it down:

Syntax Feature	YAML	JSON	Programming Metaphor
Curly braces {}	*Not used* (use indent instead)	Yes, for objects	Block/function scope, dictionaries
Arrow → or :	Uses : for key-value pairs	Uses : for key-value pairs	Maps, functions (as in (k) => v)
Parens ()	Not used	Not used	Function parameters (not in YAML)
Indentation	Defines structure	Uses braces + commas	Like Python's block scope
Comments	# comment	Not allowed (officially)	Like Python, Bash

---

📜 Example Comparison

YAML

```
person:
  name: Alice
  age: 30
  skills:
    - Azure
    - Python
```

JSON (curly braces + brackets)

```
{
  "person": {
    "name": "Alice",
    "age": 30,
    "skills": ["Azure", "Python"]
  }
}
```

Conceptual form (your () → {} idea)

```
(person) → {
  name: "Alice",
  age: 30,
  skills: ["Azure", "Python"]
}
```

This is not actual code, but a good way to **think about data** flowing from a label (key) into a structured block (value/object).

✅ **TL;DR:**

Concept	YAML
Is YAML a language?	✅ Yes, a data language (not executable)
Uses curly braces {}?	❌ No — uses indentation
Uses colons/arrow :?	✅ Yes — for key: value pairs
Uses parentheses ()?	❌ No — not function-based
Similar to?	JSON, Python dicts, INI

Would you like to see how the same data would look in YAML, JSON, and a C# or JavaScript object for full comparison?

Q - When an object serializes or deserializes that really means turning what i type in JSON stops being text in a file and becomes something that drives dot notation & IntelliSense so to speak?

Hey! Great question — and you're very close to the core idea.

Let's break it down clearly:

🔁 Serialization & Deserialization Explained

- **Serialization** = taking an object **in memory (with properties, methods, etc.)** and turning it into a format like **JSON or XML** — basically **text**.
- **Deserialization** = taking that **text (e.g., JSON)** and turning it back into an actual **object in memory**, with types, properties, and support for **dot notation** and **IntelliSense**.

✅ So yes — you're spot on:

"JSON stops being text in a file and becomes something that drives dot notation & IntelliSense"

Exactly. Here's what's happening:

- JSON:
- { "name": "TD", "age": 30 }

is just **text** — like typing into Notepad.

- When you deserialize that JSON into a C# object:
- var person = JsonSerializer.Deserialize<Person>(jsonString);
- person = { "name": "TD", "age": 30 }
- Console.WriteLine(person.age); // Dot notation!

Now person is a **strongly typed C# object**. IntelliSense kicks in because the compiler knows the structure from the Person class.

🧠 Why This Matters

- **Serialization** is used for:
  - Saving to disk
  - Sending over a network (e.g., API responses)
  - Caching objects
- **Deserialization** is used when:
  - Reading files, HTTP responses, etc.
  - Loading saved app state

- Receiving structured data (e.g., from a REST API)

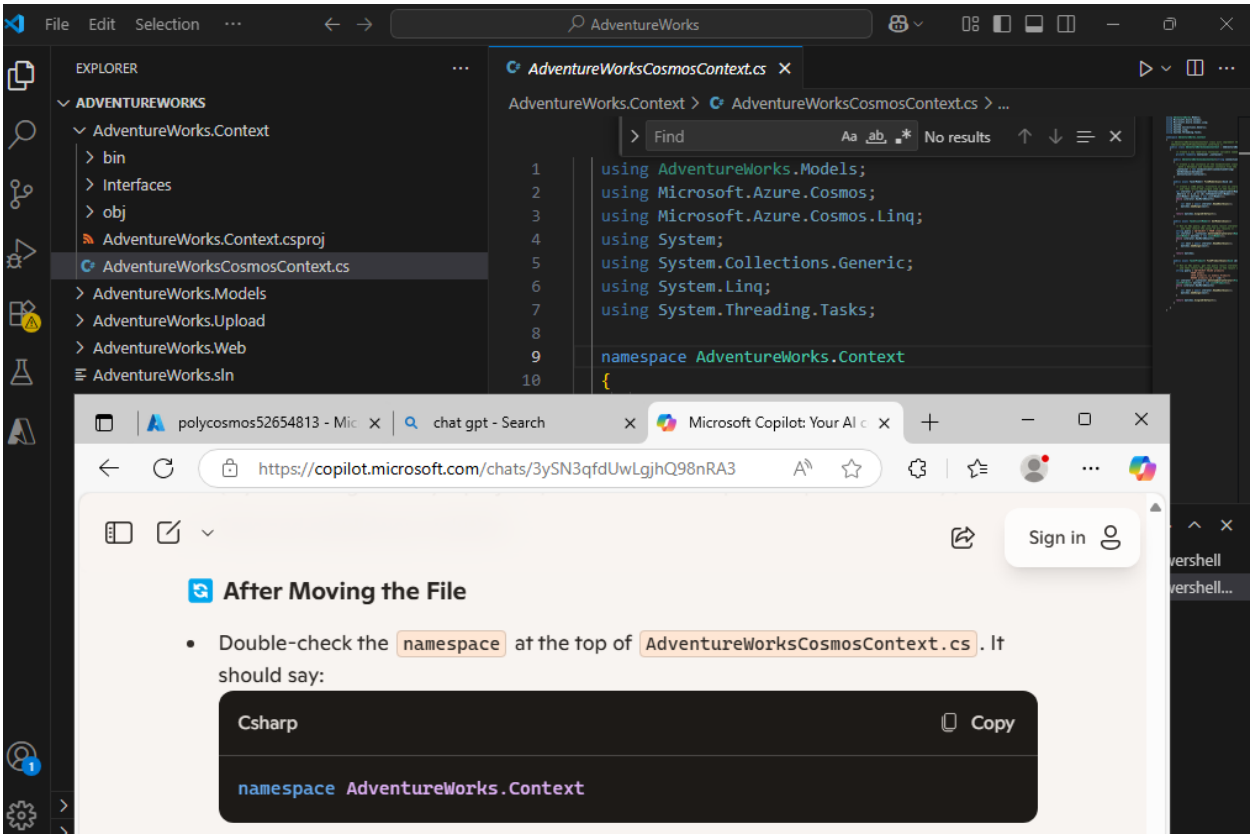
🚀 Bonus Tip

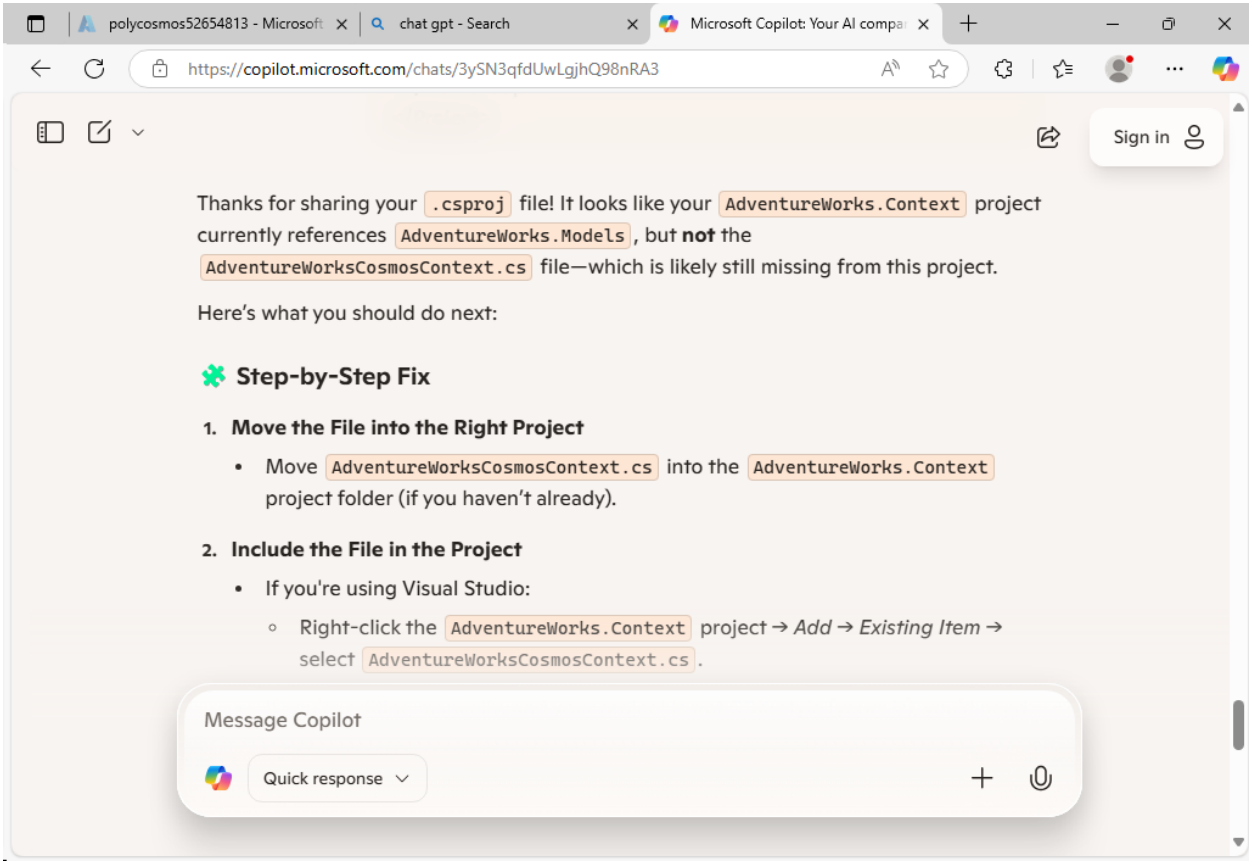
In .NET, tools like System.Text.Json or Newtonsoft.Json handle this. They "map" the JSON keys to object properties using reflection and types.

So yes — once deserialized, the object is fully usable with dot notation and IntelliSense, just like you wrote it in code.

Want a quick code example to illustrate that more directly?

Missing Reference Cosmos Lab 04





2. **Include the File in the Project**
   - If you're using Visual Studio:
     - Right-click the `AdventureWorks.Context` project → *Add* → *Existing Item* → select `AdventureWorksCosmosContext.cs`.
   - If you're editing manually, and your project is **not** SDK-style (yours is), you'd need to add:

```xml
<Compile Include="AdventureWorksCosmosContext.cs" />
```

But SDK-style projects (like yours) automatically include `.cs` files in the folder,

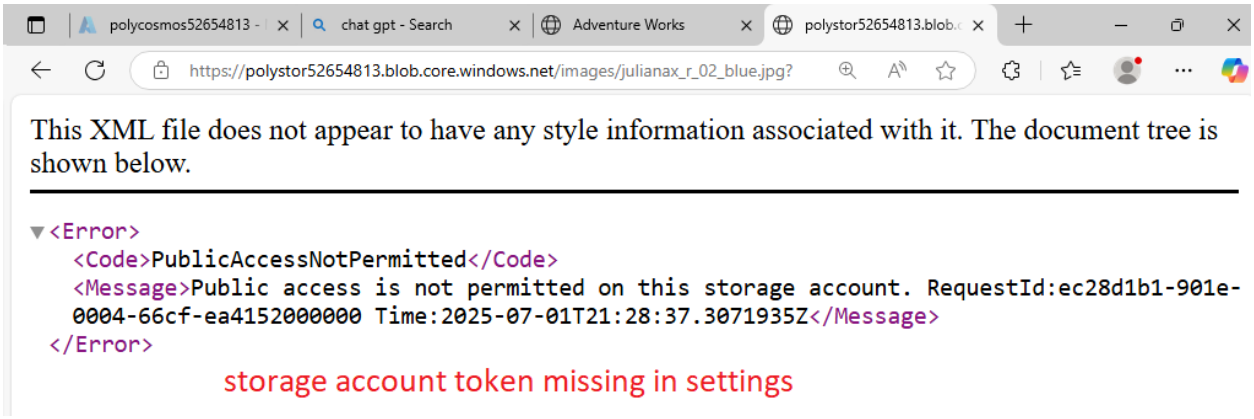
3. **Check the Namespace**
   - Open `AdventureWorksCosmosContext.cs` and make sure it starts with:

```
Csharp                                                    Copy

namespace AdventureWorks.Context
```

   - If it doesn't, either update the namespace or adjust your `using` directive in `Startup.cs`.

4. **Rebuild the Solution**
   - After moving and including the file, rebuild your solution to verify the error is resolved.

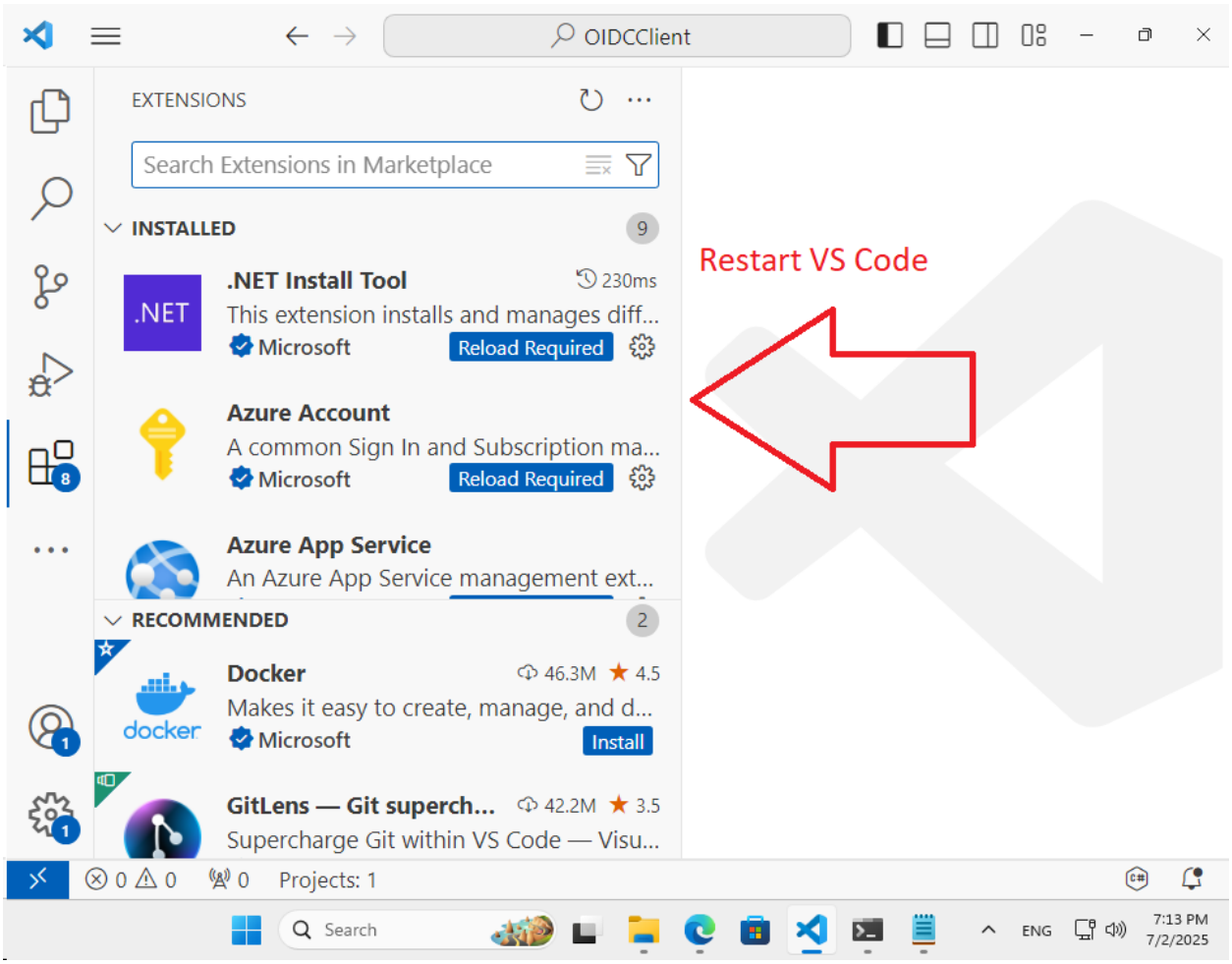


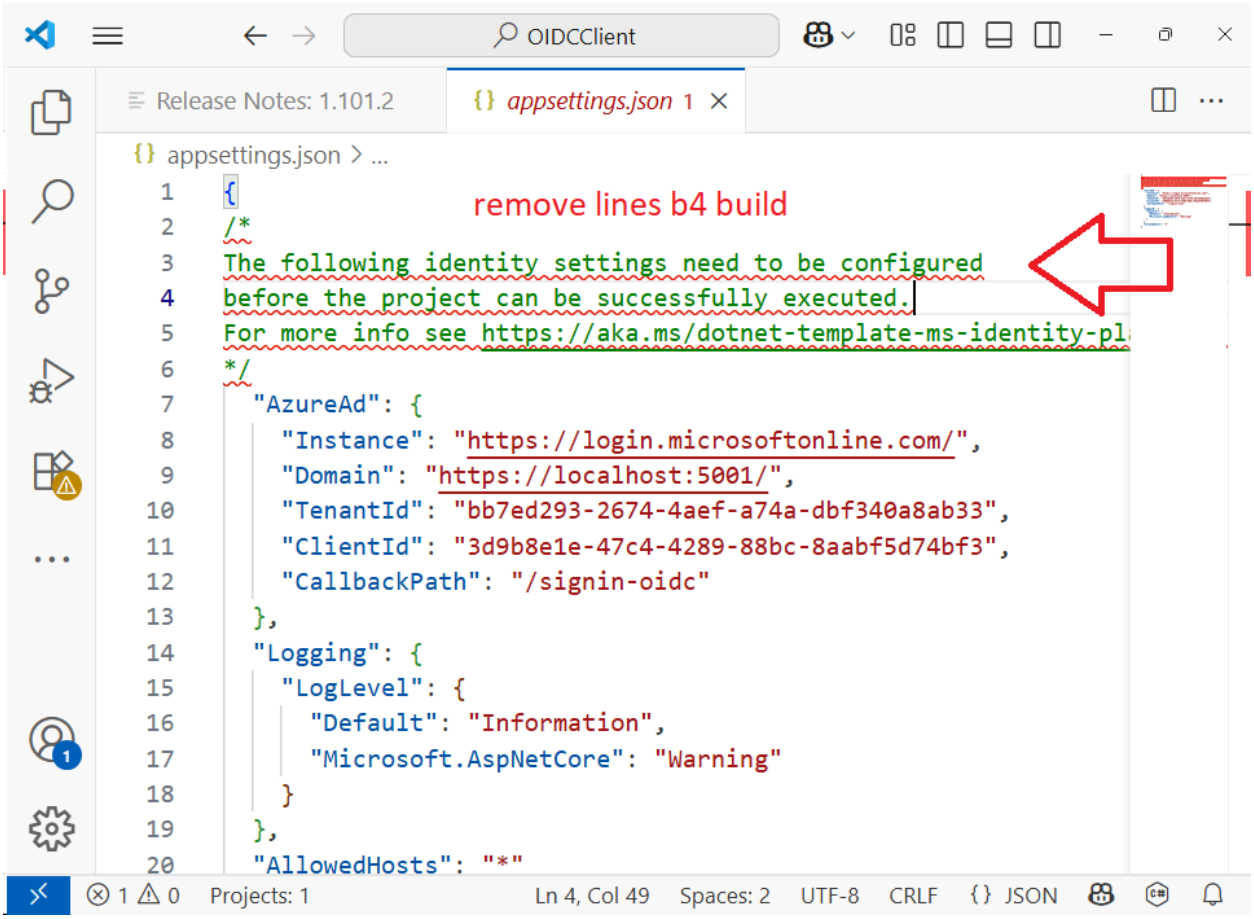
Copy & Paste Azure Cloud Shell

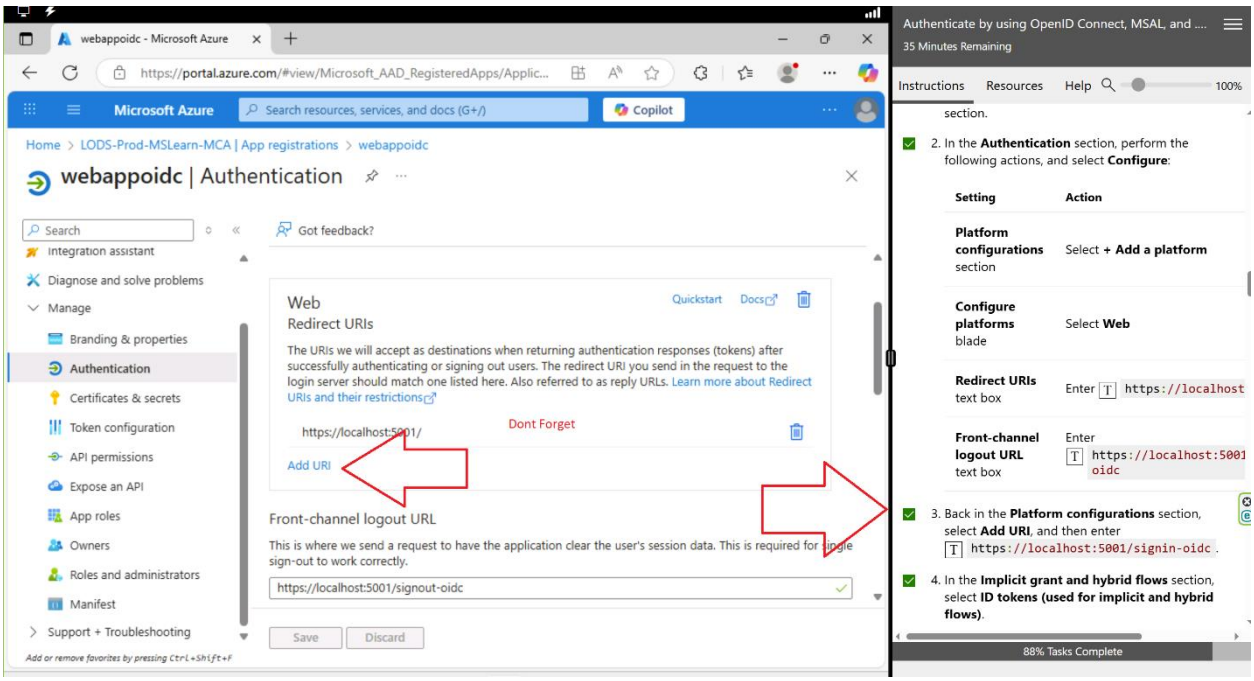
Azure Cloud Shell uses different paste shortcuts depending on your operating system.

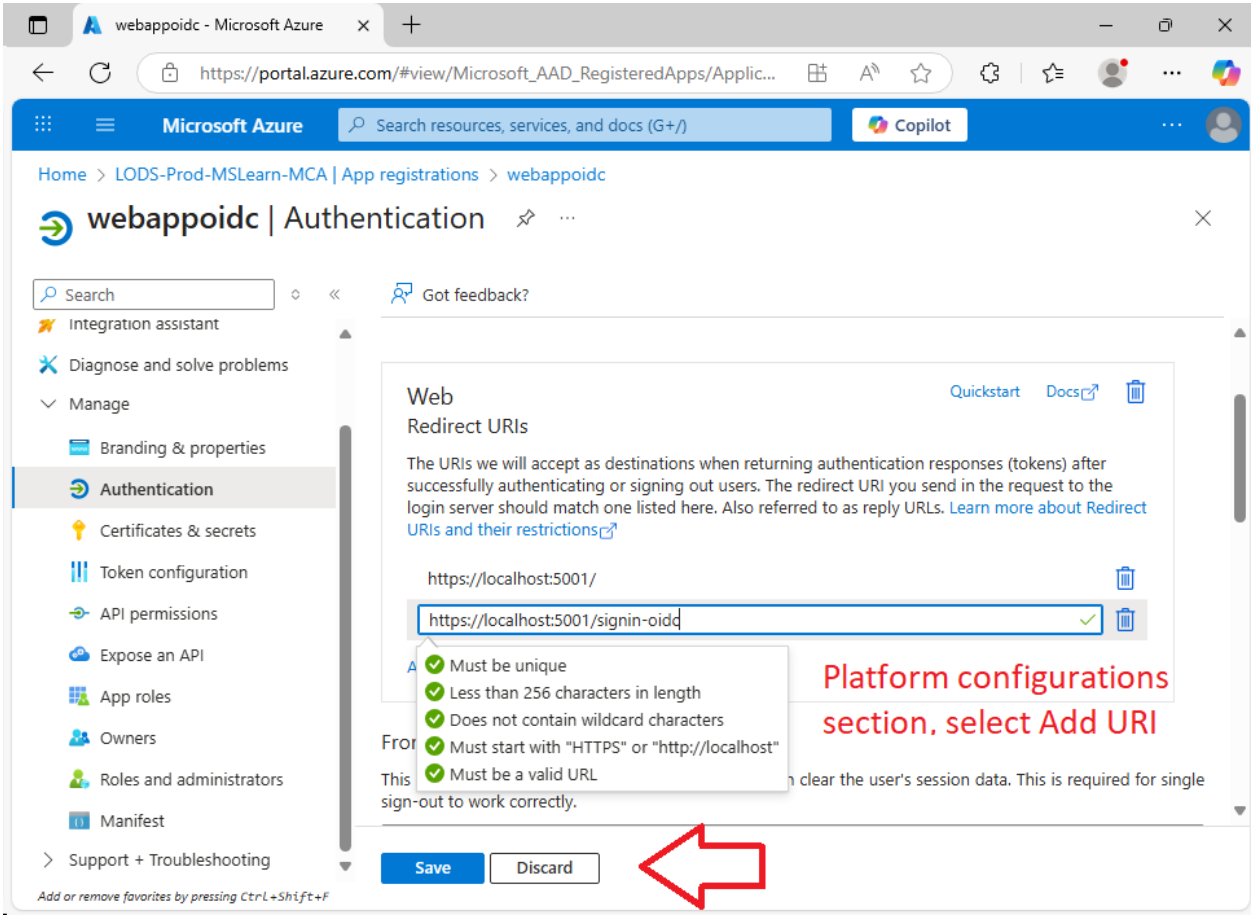
- Windows: Use Shift + Insert instead of Ctrl + V to paste. Ctrl + Shift + V is another option on some systems.
- macOS: Use Cmd + V to paste.
- Linux: Use Ctrl + Shift + V to paste.

Lab 06 OAuth Notes MS Entra ID Register




OIDCClient
Release Notes: 1.101.2
appsettings.json 1
appsettings.json > ...
remove lines b4 build
{
/*
The following identity settings need to be configured
before the project can be successfully executed.
For more info see https://aka.ms/dotnet-template-ms-identity-pl
*/
"AzureAd": {
"Instance": "https://login.microsoftonline.com/",
"Domain": "https://localhost:5001/",
"TenantId": "bb7ed293-2674-4aef-a74a-dbf340a8ab33",
"ClientId": "3d9b8e1e-47c4-4289-88bc-8aabf5d74bf3",
"CallbackPath": "/signin-oidc"
},
"Logging": {
"LogLevel": {
"Default": "Information",
"Microsoft.AspNetCore": "Warning"
}
},
"AllowedHosts": "*"
Projects: 1
Ln 4, Col 49
Spaces: 2
UTF-8
CRLF
JSON


webappoidc - Microsoft Azure
Microsoft Azure
Copilot
Home > LODS-Prod-MSLearn-MCA | App registrations > webappoidc
webappoidc | Authentication
Got feedback?
Web
Redirect URIs
Quickstart
Docs
The URIs we will accept as destinations when returning authentication responses (tokens) after successfully authenticating or signing out users. The redirect URI you send in the request to the login server should match one listed here. Also referred to as reply URLs. Learn more about Redirect URIs and their restrictions
https://localhost:5001/
Dont Forget
Add URI
Front-channel logout URL
This is where we send a request to have the application clear the user's session data. This is required for single sign-out to work correctly.
https://localhost:5001/signout-oidc
Save
Discard
Authenticate by using OpenID Connect, MSAL, and ....
35 Minutes Remaining
Instructions
Resources
Help
100%
2. In the Authentication section, perform the following actions, and select Configure:
Setting
Action
Platform configurations section
Select + Add a platform
Configure platforms blade
Select Web
Redirect URIs text box
Enter https://localhost
Front-channel logout URL text box
Enter https://localhost:5001 oidc
3. Back in the Platform configurations section, select Add URI, and then enter https://localhost:5001/signin-oidc .
4. In the Implicit grant and hybrid flows section, select ID tokens (used for implicit and hybrid flows).
88% Tasks Complete



Service Bus, Azure Queue Storage, Storage Queues, Event hub, Event Grid Comparison

Hey! Great question — Azure offers **multiple messaging services** that appear to overlap, but each is designed for **specific messaging patterns**, **features**, **delivery guarantees**, and **scalability needs**. Here's a **comparison table** with a clear definition of each:

Azure Messaging Services Comparison

Feature / Service	Azure Service Bus	Azure Queue Storage	Storage Queues (same as Queue Storage)	Azure Event Hubs	Azure Event Grid
Best For	Enterprise messaging (e.g., ordering, workflows, transactions)	Basic message queueing between components	(Same as Azure Queue Storage)	High-throughput telemetry & log ingestion	Reactive, event-driven architectures

Feature / Service	Azure Service Bus	Azure Queue Storage	Storage Queues (same as Queue Storage)	Azure Event Hubs	Azure Event Grid
Type	Message Broker (with features)	Simple Queue	(Same)	Event Streaming	Event Routing
Message Model	Messages (structured data)	Messages (text/base64)	(Same)	Streams (telemetry data)	Events (JSON, CloudEvents)
Delivery Guarantee	At least once	At least once	(Same)	At least once (some batching)	Push (at least once, retry logic)
Ordering	Supported via sessions	Not guaranteed	(Same)	Not guaranteed	Not guaranteed
Dead-lettering	✅ Yes	❌ No	(Same)	❌ No (manual)	❌ No
Duplicate Detection	✅ Yes	❌ No	(Same)	❌ No	❌ No
Advanced Features	Topics & Subscriptions, Sessions, Transactions, Filtering	None	(Same)	Partitions, Consumer Groups, Capture	Filters, event schema selection, subscriptions
Protocol	AMQP, HTTPS	HTTPS (REST)	(Same)	AMQP, HTTPS	HTTPS (webhook, hybrid connections)
Usage Pattern	**Command** (send/receive model)	**Command** (simple tasks)	(Same)	**Ingest** (big data pipelines)	**Notify** (pub-sub model)
Scale Target	High feature use, moderate throughput	High message count, low complexity	(Same)	Millions/sec throughput	Millions/day, low latency routing
Latency	Low to moderate	Low	Low	Low	Very low

Feature / Service	Azure Service Bus	Azure Queue Storage	Storage Queues (same as Queue Storage)	Azure Event Hubs	Azure Event Grid
Cost	Higher (per operation, with premium options)	Very low	(Same)	Based on throughput units (TUs)	Per million operations/events
Access	Managed identity, SAS	SAS (storage key)	(Same)	Managed identity, SAS	Managed identity, SAS

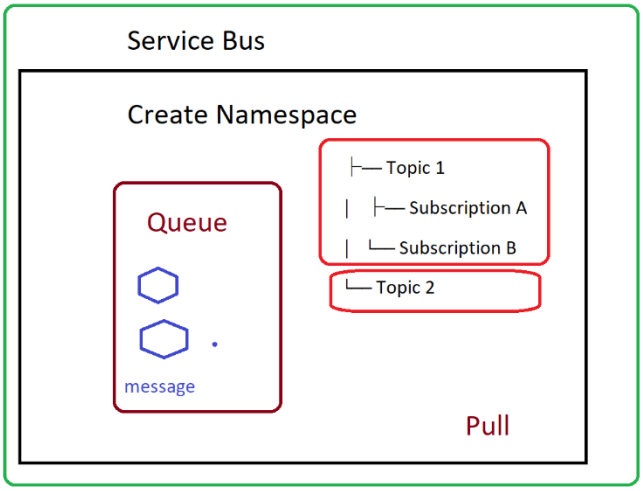
When to Use What?

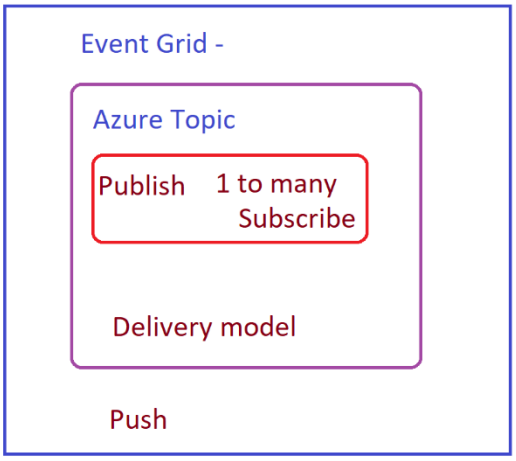
Scenario	Use This
Need **rich features** like message deferral, dead-lettering, sessions, filtering	**Azure Service Bus**
Need **simple decoupling** for background tasks, no frills	**Queue Storage**
Need to **ingest millions of telemetry messages** per second	**Event Hubs**
Need to **route events to multiple subscribers**, with filters	**Event Grid**
Want to **store queued messages in a durable, cheap way** (e.g., for retries)	**Queue Storage**

Extra Notes

- **Service Bus** supports **Topics**, enabling **1-to-many pub-sub** like Event Grid, but with full message processing and filtering features.
- **Queue Storage** is best for **simple, cheap, durable messaging** where you manage retries and failures.
- **Event Grid** is ideal for **event-based orchestration**, like notifying subscribers when a blob is uploaded.
- **Event Hubs** is designed for **big data** — firehose-style ingestion for streaming analytics platforms like Azure Stream Analytics or Databricks.

Let me know if you want example use cases or diagrams to visualize how these might work together!





Service Bus Message Unit & Billing

🔷 Is 1 Messaging Unit (MU) = 1 Message up to 1MB?

No, a **Messaging Unit (MU)** in **Azure Service Bus Premium** is **not** a per-message billing unit.

Instead, it's more like **a slice of dedicated capacity**, not a count of messages. You're **not charged per message**, but **per hour per MU**, regardless of how many messages you send — up to the throughput limits of that unit.

🔷 What *is* a Messaging Unit?

A **Messaging Unit (MU)** is a measure of **dedicated compute, memory, and I/O** capacity allocated to your **Premium Service Bus namespace**.

- 1 MU can handle:
  - **~1,000–2,000 messages/second** (depending on size and workload)
  - Messages **up to 1MB**
  - **Unlimited** message operations (within MU limits)

You **scale up** by provisioning more MUs:

- 1 MU = baseline capacity
- 2 MUs = doubled capacity
- 4 MUs = quad capacity

🧠 Think of it as **buying reserved server capacity** to process messaging workloads efficiently and with high isolation/performance.

🔷 Do You Pay More for Larger Messages (e.g., 1MB)?

Yes, **message size still matters**, even in Premium:

Message Size	Billing Impact
≤ 256 KB	1 message operation
> 256 KB (up to 1MB)	Counted as **multiple operations** (for size only in Basic/Standard)
Premium Tier	No per-message billing — **flat hourly MU fee**

So:

- **Basic/Standard** tiers will **charge multiple operations** if message size > 256 KB.
- **Premium** tier absorbs this — but you're **paying for MUs by the hour**, so large messages consume more of your allocated MU resources.

✅ Bottom Line

- **Messaging Units ≠ individual messages**
- You're **not billed per message** in Premium, but **per hour per unit**
- Premium gives you **predictable costs** and **dedicated performance**, not pay-as-you-go per operation like Basic/Standard

Would you like a calculator or sample scenario for choosing MUs based on expected traffic or size?

Yes — you're spot on. Let's break it down cleanly:

🔷 Is a Namespace the Parent to Queues and Topics?

✅ **Yes**, in Azure Service Bus, a **Namespace** is the **top-level container**.

Think of it like this:

Service Bus Namespace

├— Queue 1

├— Queue 2

├— Topic 1

| ├— Subscription A

| └── Subscription B

└── Topic 2

A **namespace** holds:

- **Queues**
- **Topics** (and their Subscriptions)
- Shared settings like:
    - Authorization rules (Access Control)
    - Metrics
    - Geo-replication (if Premium)

🔷 How Many Namespaces or Queues Can You Have?

Here are the **limits** (as of current Azure docs):

Entity	Limit
Namespaces per Azure Subscription	~1000 (soft limit; can request more)
Queues per Namespace	**10,000**
Topics per Namespace	**10,000**
Subscriptions per Topic	**2,000**
Rules per Subscription	100

💡 Note: Limits may differ slightly between **tiers (Basic, Standard, Premium)** and some can be increased via Azure Support.

🔷 Can You Have Multiple Namespaces?

✅ Yes, and you often **should**:

- For **isolation** (e.g., dev/test/prod)
- To avoid **limit collisions**
- To deploy in **different regions**
- For **multi-tenancy** or performance scaling

Summary

Term	Role	Notes
Namespace	Container for all entities	Think of it like a messaging "server"
Queue	Holds messages (1:1)	FIFO, supports sessions, DLQs
Topic	Pub-sub messaging (1:many)	With filters and rules
Subscription	Listener to a topic	Can filter which messages it gets

Would you like a visual diagram or quick example of a real-world layout?

That's a sharp observation — and you're on the right conceptual track — but **Azure Event Grid Topics** and **Azure Service Bus Namespaces** are actually **entirely separate services** under the hood, even if they **seem similar** in structure.

Let's clarify the overlap and key differences:

🔷 **Are Event Grid Topics and Service Bus Namespaces the same?**

❌ **No. They are different services, with different goals and infrastructure.**

Feature	Azure Event Grid	Azure Service Bus
Namespace concept?	❌ No namespace — just topics	✅ Has a namespace containing queues/topics
Messaging model	**Push-based** eventing	**Pull-based** messaging queue/topic model
Topic =	Custom event publisher (like a webhook hub)	Pub-sub message router inside a namespace
Delivery	Push to endpoints (HTTP, Function, Logic App)	Pull (or peek-lock) from queue/subscription
Latency	Near real-time (low latency)	Optimized for reliability & order guarantees
Durability	Best effort (24-hour retry, default 1–3 days)	Persistent with dead-lettering and sessions
Fan-out behavior	Native (1:N via subscriptions)	Via Topics + Subscriptions

---

🔷 **Why it Feels Similar**

You're right that **Event Grid Topics** resemble **Service Bus Topics**:

- Both are **publish-subscribe** models.
- Both have **subscriptions** (Event Grid Subscriptions vs Service Bus Topic Subscriptions).
- Both **route messages/events** to multiple handlers.

But the core difference is **intent and infrastructure**:

Comparison	Event Grid Topic	Service Bus Topic
Designed for	Event-driven architectures	Messaging with strict control
Subscription model	Push to endpoint	Queue-like pull model
Retry behavior	At-least-once, timed expiry	At-least-once, persisted
Filtering	Event filters by type, subject	SQL-like rules on subscriptions

---

🔷 **So What's the Takeaway?**

- **Event Grid Topic**: Lightweight, reactive, web-native event publisher.
- **Service Bus Namespace**: Heavy-duty, enterprise-grade messaging broker.

✅ **Both support pub/sub**, but:

- Event Grid is **event distribution**
- Service Bus is **message routing & persistence**

Let me know if you want a table comparing **Event Grid**, **Event Hub**, and **Service Bus** — that's where a lot of confusion stems from!

Great question — **Azure Event Hubs** is the third player in Azure's messaging/eventing trio, and it fills a **high-throughput, streaming** role that sits **between Event Grid and Service Bus** in terms of purpose.

Here's a clean breakdown of how **Event Hubs** fits into the mix:

🔺 **Azure Messaging/Events Services Summary**

Feature	Event Grid	Event Hubs	Service Bus
Purpose	Event routing (notifications)	Real-time **event streaming/logs**	Reliable enterprise **messaging**

Feature	Event Grid	Event Hubs	Service Bus
Pattern	Push (1:N fan-out)	Ingest and stream (1:many consumers)	Pull/push queue/topic-based
Delivery	Push to endpoints (HTTP)	Pull by consumers (Kafka-like)	Pull/peek-lock by receivers
Latency	Low (milliseconds)	Low (ms to seconds)	Medium (configurable)
Retention	Short (1–3 days by default)	**Configurable (up to 90 days)**	Persistent until read
Ordering	No (unless 1 topic & single event handler)	Yes (per partition)	Yes (FIFO, sessions supported)
Scale	Millions of events/sec	Millions/sec per partition	Thousands/sec
Use Case	Lightweight pub-sub	Big data ingestion, telemetry, logs	Business logic integration, workflows
Consumer Model	Event handlers (push)	Stream processors (pull via offset)	Clients/apps consuming messages
Built-in Retry	Yes, 24–72h delivery attempts	Checkpointing & replays	DLQ, retries, transactions

What Exactly Is Event Hubs?

✅ Event Hubs is:

- A **high-throughput event ingestion pipeline**
- Modeled like **Apache Kafka**
- Good for **real-time analytics**, telemetry, IoT, log ingestion

It supports:

- **Partitioned** message storage
- **Offset-based** read (like a durable stream)
- **Capture** to Azure Blob/Data Lake for archiving
- Integration with tools like:
  - Azure Stream Analytics
  - Apache Spark
  - Kafka-compatible clients
  - Azure Functions (via bindings)

When to Use What?

Scenario	Best Choice
IoT telemetry ingestion	✅ **Event Hubs**
Audit logs / telemetry to storage	✅ **Event Hubs**
"Something happened" notification	✅ **Event Grid**
Email / payment processing queue	✅ **Service Bus**
Ordered, transactional workflows	✅ **Service Bus**
Distributed pub-sub w/ filtering	✅ **Event Grid**

TL;DR Mental Model

Service	Like...	Analogy
Event Grid	Email notification system	"Notify me when X happens"
Event Hubs	Log file or Kafka pipeline	"Stream every change for later"
Service Bus	Enterprise mailroom	"Deliver messages reliably"

Let me know if you want a **diagram** showing how all three interconnect or an **example solution** combining all of them — that's a powerful pattern!

Azure Storage Queues vs. Service Bus Queues

Offer different features and are suitable for different scenarios. Storage queues are simpler, more cost-effective, and good for basic asynchronous message passing. Service Bus queues offer more advanced features like pub/sub, transactions, and complex routing, making them suitable for more sophisticated messaging scenarios.

Compare Azure Storage queues and Service Bus queues - Azure Service Bus | Microsoft Learn

Containers Including Docker

1. Containers in Visual Studio Code
2. Get started with container apps in Visual Studio Code | Microsoft Learn
3. A container is a process on your computer. It's isolated from all other processes on the host computer. That isolation uses kernel namespaces and control groups.
4. A container uses an isolated filesystem. This custom filesystem is provided by a container image. The image contains everything needed to run an application, such as all dependencies, configuration, scripts, and binaries
5. Tutorial - Quick Docker Image Build with Azure ACR Tasks - Azure Container Registry | Microsoft Learn

Azure Container Registry Tasks (ACR Tasks)

6. Automate Container Builds with Azure Container Registry Tasks - Azure Container Registry | Microsoft Learn
    a. Enable automated builds triggered by source code updates, updates to a container's base image, or timers.
7. Sample Azure Container Registry (ACR) Tasks - Azure Container Registry | Microsoft Learn

Great question, Tim! You're absolutely right that both **Azure Container Instances (ACI)** and **Azure Container Apps (ACA)** pull container images from a registry (like Azure Container Registry or Docker Hub), similar to how you'd copy files to a web server before serving them. But the big difference lies in **what happens after the image is pulled**—the runtime environment and orchestration model.

Here's the breakdown:

☑ Azure Container Instances (ACI)

- **What it is:** ACI is a lightweight service for running containers directly on Azure without managing VMs or orchestrators.
- **How it runs containers:**
    - It spins up an isolated container (or a small group in a container group) on Azure's infrastructure.
    - There's **no orchestration layer**—each container group is standalone.
- **Use case:**
    - Great for short-lived tasks, batch jobs, or simple microservices.
- **Scaling:**
    - Manual or via external automation (e.g., Logic Apps, Functions). No built-in autoscaling.
- **Networking:**
    - Basic networking options, but not as advanced as ACA.

☑ Azure Container Apps (ACA)

- **What it is:** ACA is a **managed container orchestration platform** built on Kubernetes and Envoy, but abstracted so you don't manage clusters.
- **How it runs containers:**
    - Containers run inside a **Kubernetes-based environment** with features like revisions, traffic splitting, and Dapr integration.
    - ACA adds **autoscaling (including KEDA)**, secrets management, and service discovery.
- **Use case:**
    - Ideal for microservices, event-driven apps, and apps needing scale-to-zero.
- **Networking:**
    - Advanced networking, ingress, and service-to-service communication.

🔑 **Key Difference**

Think of ACI as "just run this container now," while ACA is "run this container as part of a scalable, orchestrated application."
Both pull images from a registry, but ACA adds orchestration, scaling, and app lifecycle management.

Would you like me to **draw a quick diagram comparing ACI vs ACA architecture** (showing where the container image fits in and what extra layers ACA adds)? Or should I **write a concise analogy comparing them to web hosting models**?

Deployment Slots Swapping

Swap deployment slots in Azure App Service | Develop solutions for Microsoft Azure

Reference

1. Overview | Dapr Docs
    a. Incrementally migrate your existing applications to a microservices architecture, thereby adopting cloud native patterns such scale out/in, resiliency, and independent deployments.
2. Sign in with Azure CLI — Login and Authentication | Microsoft Learn
3. Install the Azure CLI for Windows | Microsoft Learn
4. Routing traffic & View Cookies in Client Browser
5. View, edit, and delete cookies - Microsoft Edge Development | Microsoft Learn